

```

1. =====
2. # SENTINEL v4 KERNEL — MINIMAL UNIFIED ARCHITECTURE
3. # =====
4.
5. from __future__ import annotations
6. from dataclasses import dataclass, field
7. from typing import Dict, List, Tuple, Optional
8. import math
9. import random
10. import copy
11.
12. # =====
13. # 1. EPISYSTEM (Observer–System Coupling Core)
14. # =====
15.
16. class Episystem:
17.     """
18.     Second-order cybernetics + chaos + observer coupling.
19.     Observation is part of system dynamics.
20.     """
21.
22.     def __init__(self):
23.         self.observer_influence = 0.05
24.         self.prediction_horizon = 50
25.         self.reflexivity_index = 0.0
26.
27.     def apply_observation_feedback(self, state: Dict, signal: Dict):
28.         # observation changes system slightly (second-order effect)
29.         for k, v in signal.items():
30.             if k in state:
31.                 state[k] += self.observer_influence * (v - state[k])
32.         return state
33.
34.     def compute_reflexivity(self, error_history: List[float]) -> float:
35.         if not error_history:
36.             return 0.0
37.         drift = sum(error_history[-10:]) / len(error_history[-10:])
38.         self.reflexivity_index = min(1.0, abs(drift))
39.         return self.reflexivity_index
40.
41.     def chaos_horizon_decay(self, sensitivity: float) -> int:
42.         # exponential loss of predictability
43.         return max(1, int(self.prediction_horizon * math.exp(-sensitivity)))
44.

```

```

45.
46. # =====
47. # 2. CONTROL TOPOLOGY LAYER (VSM + Governance)
48. # =====
49.
50. class ControlTopology:
51. """
52. VSM-style recursive control architecture.
53. """
54.
55. def __init__(self):
56. self.S1 = {}
57. self.S2 = {}
58. self.S3 = {}
59. self.S4 = {}
60. self.S5 = {
61. "fairness_weight": 1.0,
62. "stability_priority": 1.0,
63. "efficiency_priority": 1.0
64. }
65.
66. def coordinate(self, state: Dict) -> Dict:
67. # S2 coordination (friction reduction)
68. return {k: v * 0.98 for k, v in state.items()}
69.
70. def optimize(self, state: Dict) -> Dict:
71. # S3 optimization (simple smoothing)
72. return {k: (v + 0.1) / 1.1 for k, v in state.items()}
73.
74. def forecast(self, state: Dict) -> Dict:
75. # S4 simple projection
76. return {k: v * 1.02 for k, v in state.items()}
77.
78. def apply_governance(self, state: Dict) -> Dict:
79. # S5 fairness constraint (Rawls-like maximin bias)
80. min_val = min(state.values()) if state else 0
81. return {k: min(v, min_val * self.S5["fairness_weight"] + v * 0.5)
82. for k, v in state.items()}
83.
84.
85. # =====
86. # 3. DYNAMICAL FIELD ENGINE (Reservoir + Chaos + CA)
87. # =====
88.

```

```

89. class DynamicalField:
90. """
91. Reservoir-style nonlinear state space with chaotic evolution.
92. """
93.
94. def __init__(self, size: int = 10):
95. self.state = [random.random() for _ in range(size)]
96. self.size = size
97.
98. def step(self, input_signal: List[float]) -> List[float]:
99. new_state = []
100. for i in range(self.size):
101. left = self.state[i - 1]
102. center = self.state[i]
103. right = self.state[(i + 1) % self.size]
104. inp = input_signal[i % len(input_signal)]
105.
106. # nonlinear chaotic update
107. val = math.tanh(left + center * 0.5 + right * 0.2 + inp)
108. new_state.append(val)
109.
110. self.state = new_state
111. return new_state
112.
113. def embed(self) -> List[float]:
114. # reservoir-style nonlinear projection
115. return [math.tanh(x * random.uniform(0.8, 1.2)) for x in self.state]
116.
117.
118. # =====
119. # 4. FLOW & STRUCTURAL EVOLUTION ENGINE
120. # =====
121.
122. class FlowEngine:
123. """
124. Constructal law + mycorrhizal redistribution logic.
125. """
126.
127. def __init__(self):
128. self.flow_graph = {"A": 1.0, "B": 1.0, "C": 1.0}
129.
130. def update_flow(self, demand: Dict[str, float]):
131. # redistribute capacity toward high demand
132. total = sum(demand.values()) + 1e-6

```

```

133. for k in self.flow_graph:
134.     target = demand.get(k, 0.1) / total
135.     self.flow_graph[k] += 0.1 * (target - self.flow_graph[k])
136.
137. def bottleneck_score(self) -> float:
138.     vals = list(self.flow_graph.values())
139.     return max(vals) - min(vals)
140.
141.
142. # =====
143. # 5. REGIME & BEHAVIORAL INTELLIGENCE
144. # =====
145.
146. class RegimeEngine:
147.     """
148.     Regime detection + stability + simple game-theoretic signals.
149.     """
150.
151.     def detect_regime(self, state: Dict) -> str:
152.         avg = sum(state.values()) / len(state)
153.         if avg < 0.3:
154.             return "STABLE"
155.         elif avg < 0.6:
156.             return "SURGE"
157.         else:
158.             return "PANIC"
159.
160.     def stability_score(self, state: Dict) -> float:
161.         vals = list(state.values())
162.         return 1.0 - (max(vals) - min(vals))
163.
164.
165. # =====
166. # 6. ETHICAL-INVARIANT GOVERNOR (Autopoiesis + Rawls)
167. # =====
168.
169. class EthicalGovernor:
170.     """
171.     Ensures identity preservation + worst-case fairness constraint.
172.     """
173.
174.     def __init__(self):
175.         self.identity_signature = "SENTINEL_V4"
176.

```

```

177. def rawls_filter(self, state: Dict) -> Dict:
178.     # maximin constraint
179.     min_val = min(state.values()) if state else 0
180.     return {k: max(v, min_val) for k, v in state.items()}
181.
182. def autopoietic_check(self, system_state: Dict) -> bool:
183.     # system must retain structural coherence
184.     return len(system_state) > 0 and all(v is not None for v in system_state.values())
185.
186.
187. # =====
188. # SENTINEL V4 ORCHESTRATOR
189. # =====
190.
191. class SentinelV4:
192.
193.     def __init__(self):
194.         self.episystem = Episystem()
195.         self.control = ControlTopology()
196.         self.dynamics = DynamicalField()
197.         self.flow = FlowEngine()
198.         self.regime = RegimeEngine()
199.         self.ethics = EthicalGovernor()
200.
201.         self.error_history = []
202.
203.     def step(self, input_signal: Dict[str, float]) -> Dict:
204.
205.         # --- 1. Dynamics engine (core state evolution)
206.         dyn_input = list(input_signal.values())
207.         dyn_state = self.dynamics.step(dyn_input)
208.         state = {f"x{i}": v for i, v in enumerate(dyn_state)}
209.
210.         # --- 2. Flow evolution
211.         self.flow.update_flow(input_signal)
212.         state["bottleneck"] = self.flow.bottleneck_score()
213.
214.         # --- 3. Control topology
215.         state = self.control.coordinate(state)
216.         state = self.control.optimize(state)
217.         state = self.control.forecast(state)
218.         state = self.control.apply_governance(state)
219.
220.         # --- 4. Ethical constraint layer

```

```

221. state = self.ethics.rawls_filter(state)
222. assert self.ethics.autopoietic_check(state)
223.
224. # --- 5. Regime detection
225. regime = self.regime.detect_regime(state)
226. stability = self.regime.stability_score(state)
227.
228. # --- 6. Epistemic feedback (second-order effect)
229. state = self.epistemic.apply_observation_feedback(state, input_signal)
230.
231. # simulate prediction error (toy)
232. error = abs(sum(state.values()) - sum(input_signal.values()))
233. self.error_history.append(error)
234.
235. reflexivity = self.epistemic.compute_reflexivity(self.error_history)
236.
237. return {
238.     "state": state,
239.     "regime": regime,
240.     "stability": stability,
241.     "reflexivity": reflexivity,
242.     "flow": copy.deepcopy(self.flow.flow_graph),
243. }
244.
245.
246. # =====
247. # SIMPLE RUN LOOP (EXAMPLE)
248. # =====
249.
250. if __name__ == "__main__":
251.     sentinel = SentinelV4()
252.
253.     for t in range(10):
254.         input_signal = {
255.             "aht": random.random(),
256.             "abandon": random.random(),
257.             "queue": random.random()
258.         }
259.
260.         output = sentinel.step(input_signal)
261.         print(f"\nSTEP {t}")
262.         print(output)
263.
264.

```

```
265.
266.
267.
268. #
=====
269. # SENTINEL - UNIFIED ARCHITECTURE
270. #
=====
271. #
272. # SENTINEL SYSTEM OBJECTIVE
273. #
274. # Sentinel is a resilience-oriented synthetic IVR
275. # control, simulation, orchestration, and evaluation
276. # environment designed to maintain trustworthy,
277. # stable, adaptive, and non-exploitable borrower
278. # interaction behavior during large-scale borrower
279. # stress events.
280. #
281. # The system is specifically intended to model,
282. # simulate, evaluate, stress-test, and stabilize
283. # borrower interaction systems during repayment
284. # disruptions and operational instability events such as:
285. #
286. # - SAVE repayment restart surges
287. # - payment shock waves
288. # - borrower confusion amplification
289. # - queue saturation cascades
290. # - fraud exploitation attempts
291. # - misinformation propagation
292. # - operational instability feedback loops
293. # - recursive IVR traversal behavior
294. # - emotional escalation patterns
295. #
296. # PRIMARY OBJECTIVE
297. #
298. # Preserve borrower-system stability during
299. # high-volatility operational stress without
300. # allowing optimization pressure to create:
301. #
302. # - borrower confusion amplification
303. # - recursive instability loops
```

```
304. # - exploit-friendly deterministic pathways
305. # - queue collapse propagation
306. # - psychological escalation
307. # - instability masking
308. # - operational fragility
309. # - misinformation cascades
310. #
311. # SYSTEM PHILOSOPHY
312. #
313. # Sentinel does NOT optimize purely for:
314. #   containment / AHT reduction / automation maximization /
315. #   transfer suppression / queue minimization / local KPI
    optimization
316. #
317. # Instead, Sentinel performs GOVERNED OPTIMIZATION,
318. # where adaptation is constrained by:
319. #   resilience preservation / borrower comprehension /
320. #   psychological safety / exploit resistance /
321. #   regime stability / operational trustworthiness / systemic
    integrity
322. #
323. # CORE DESIGN PRINCIPLES
324. #   1. Stability over raw efficiency
325. #   2. Trust preservation over containment
326. #   3. Adaptive orchestration over static routing
327. #   4. Regime-aware behavior over fixed logic
328. #   5. Controlled learning over unrestricted optimization
329. #   6. Exploit resistance over deterministic repeatability
330. #   7. System resilience over local KPI maximization
331. #   8. Temporal stability over short-term optimization
332. #   9. Borrower comprehension over automation depth
333. #   10. Cascading-failure prevention over throughput gains
334. #
335. # PROTECTED CONSTRAINTS
336. # Sentinel must avoid: forced containment / borrower dead-end
    loops /
337. # artificial call shortening / exploit amplification /
338. # escalation suppression / instability masking / misleading
    routing /
339. # recursive confusion traps / unsafe autonomous optimization
    /
```



```

340. # queue-preservation behaviors that reduce clarity / metrics
    gaming
341. #
342. # SUCCESS CONDITION
343. # Operational stability, borrower comprehension, queue
    integrity,
344. # and exploit resistance maintained during stress-regime
    transitions
345. # without recursive instability or sacrificing systemic
    trustworthiness.
346. #
347. # FAILURE CONDITION
348. # Optimization improves local metrics while simultaneously
    increasing:
349. # borrower confusion / exploitability / operational fragility
    /
350. # instability propagation / recursive call behavior /
351. # misinformation / psychological escalation / queue collapse
    risk
352. #
353. # PERSISTENT EXPORT INSTRUCTIONS
354. # ALL future exports MUST: include this header, include the
    cumulative
355. # change log, preserve architectural intent, preserve
    protected
356. # constraints, append new modifications chronologically,
    never optimize
357. # local metrics in ways that violate global resilience
    objectives.
358. #
359. #
    =====
360.
361. from __future__ import annotations
362.
363. import copy
364. import csv
365. import hashlib
366. import json
367. import logging
368. import math
369. import random

```

```

370. import statistics
371. import time
372. from collections import deque
373. from dataclasses import dataclass, field
374. from datetime import datetime
375. from enum import Enum
376. from typing import (
377.     Any, Callable, Dict, Generator, List, Optional, Tuple
378. )
379.
380. import numpy as np
381. import matplotlib
382. matplotlib.use("Agg")
383. import matplotlib.pyplot as plt
384.
385.
386. #
=====
387. # LOGGING
388. #
=====
389.
390. logger = logging.getLogger("Dual Harness")
391. logger.setLevel(logging.INFO)
392.
393. if not logger.handlers:
394.     _handler = logging.StreamHandler()
395.     _handler.setFormatter(logging.Formatter("[Dual Harness]
    %(message)s"))
396.     logger.addHandler(_handler)
397.
398.
399. #
=====
400. # PART A - MATH UTILITIES (Module 1)
401. #
=====
402.
403. def logit(p: float) -> float:
404.     """ln(p / (1-p)) - converts probability to log-odds."""
405.     p = min(max(p, 1e-9), 1 - 1e-9)

```

```

406.         return math.log(p / (1 - p))
407.
408.
409.     def inv_logit(x: float) -> float:
410.         """Logistic sigmoid – converts log-odds back to
            probability."""
411.         return 1.0 / (1.0 + math.exp(-x))
412.
413.
414.     def soft_bound_sigmoid(x: float) -> float:
415.         """Smoothly maps any real value to [0, 1]."""
416.         return 1.0 / (1.0 + math.exp(-x))
417.
418.
419.     #
            =====
420.     # PART B – ENUMS
421.     #
            =====
422.
423.     class PopulationType(Enum):
424.         NORMAL = "normal"
425.         SAVE   = "save"
426.         FRAUD  = "fraud"
427.
428.
429.     class SentinelResult(Enum):
430.         NEUTRAL      = "NEUTRAL"
431.         REGRESSIVE   = "REGRESSIVE"
432.         RISK_INCREASING = "RISK_INCREASING"
433.
434.
435.     class SystemRegime(Enum):
436.         STABLE      = "stable"
437.         SURGE       = "surge"
438.         CONFUSION   = "confusion"
439.         PANIC       = "panic"
440.         SATURATED   = "saturated"
441.         FRAUD_WAVE  = "fraud_wave"
442.
443.

```

```

444. class TouchpointChannel(Enum):
445.     IVR          = "ivr"
446.     EMAIL        = "email"
447.     WEB_PORTAL   = "web_portal"
448.     LETTER       = "direct_mail"
449.     SMS          = "sms"
450.
451.
452. #
=====
453. # PART C - CONFIGURATION
454. #
=====
455.
456. LEARNING_EXCLUSIONS: set = {
457.     "breakalignedabandons",
458.     "atoeveningsaturation",
459.     "hostileautomation",
460.     "ivrportsaturation",
461. }
462.
463. HYSTERESIS_MARGIN          = 0.12
464. BACKOFFICE_REGRESSION_THRESHOLD = 0.02
465. FRAUD_DETERMINISM_THRESHOLD = 0.05
466. VELOCITY_MAX_BOUND        = 2.5
467.
468. PRIORITY_PROFILES: Dict[str, List[str]] = {
469.     "stable": ["operationalefficiency",
470.               "futureselfresolution",
471.               "informationaccuracy",
472.               "borrowerpsychologicalsafety"],
473.     "surge": ["futureselfresolution",
474.               "borrowerpsychologicalsafety",
475.               "informationaccuracy",
476.               "operationalefficiency"],
477.     "confusion": ["informationaccuracy",
478.                   "borrowerpsychologicalsafety",
479.                   "futureselfresolution"],
480.     "panic": ["borrowerpsychologicalsafety",
481.               "queuepreservation",
482.               "informationaccuracy"],

```

```

477.     "saturated": ["queuepreservation",
478.     "borrowerpsychologicalsafety"],
479. }
480.
481. # Regime-aware tolerance matrices (restored from v2
    hardening)
482. REGIME_TOLERANCE_MATRICES: Dict[SystemRegime, Dict[str,
    float]] = {
483.     SystemRegime.STABLE: {"max_aht_delta": 20.0,
484.     "max_callback_delta": 0.03,
485.     "max_abandon_delta": 0.02,
486.     "min_containment_delta": -0.03,
487.     "max_queue_depth": 0.80},
488.     SystemRegime.SURGE: {"max_aht_delta": 60.0,
489.     "max_callback_delta": 0.08,
490.     "max_abandon_delta": 0.05,
491.     "min_containment_delta": -0.08,
492.     "max_queue_depth": 0.95},
493.     SystemRegime.CONFUSION: {"max_aht_delta": 40.0,
494.     "max_callback_delta": 0.05,
495.     "max_abandon_delta": 0.04,
496.     "min_containment_delta": -0.04,
497.     "max_queue_depth": 0.85},
498.     SystemRegime.PANIC: {"max_aht_delta": 90.0,
499.     "max_callback_delta": 0.12,
500.     "max_abandon_delta": 0.08,
501.     "min_containment_delta": -0.15,
502.     "max_queue_depth": 0.98},
503.     SystemRegime.SATURATED: {"max_aht_delta": 30.0,
504.     "max_callback_delta": 0.06,
505.     "max_abandon_delta": 0.06,
506.     "min_containment_delta": -0.05,
507.     "max_queue_depth": 0.99},
508.     SystemRegime.FRAUD_WAVE: {"max_aht_delta": 15.0,
509.     "max_callback_delta": 0.02,
510.     "max_abandon_delta": 0.02,
511.     "min_containment_delta": -0.02,
512.     "max_queue_depth": 0.80},
513. }

```

```

503. # Lyapunov energy weights (mutable for auto-tuning)
504. LYAPUNOV_ENERGY_WEIGHTS: Dict[str, float] = {
505.     "abandon_rate": 1.0,
506.     "ivr_reentry_rate": 1.0,
507.     "transfer_rate": 1.0,
508.     "normalized_aht": 1.0,
509. }
510.
511.
512. #
=====
513. # PART D - CORE DATA MODELS
514. #
=====
515.
516. @dataclass
517. class RatePair:
518.     """Paired probability: unclipped raw value + bounded
    final value."""
519.     unclipped: float
520.     p: float
521.
522.
523. @dataclass
524. class Metrics:
525.     """Full operational metrics used across all
    subsystems."""
526.     containment: float = 0.0
527.     aht: float = 0.0
528.     callback_rate: float = 0.0
529.     abandon_rate: float = 0.0
530.     backoffice_trigger: float = 0.0
531.     determinism_index: float = 1.0
532.
533.     repeat_auth_rate: float = 0.0
534.     ivr_reentry_rate: float = 0.0
535.     transfer_rate: float = 0.0
536.     short_disconnect_rate: float = 0.0
537.     queue_depth: float = 0.0
538.
539.     # Diagnostics

```

```

540.         unclipped_abandon_rate: Optional[float] = None
541.         unclipped_callback_rate: Optional[float] = None
542.         unclipped_ivr_reentry_rate: Optional[float] = None
543.
544.
545.     @dataclass
546.     class PopulationResult:
547.         population: PopulationType
548.         result: SentinelResult
549.         rationale: str
550.         dominant_regime: SystemRegime
551.         regime_confidence: float
552.         regime_scores: Dict[str, float]
553.         run_id: str
554.         timestamp: datetime
555.
556.
557.     @dataclass
558.     class TemporalWindow:
559.         recent_metrics: List[Metrics] =
560.             field(default_factory=list)
561.
562.         def rolling_mean(self, attr: str) -> float:
563.             values = [getattr(m, attr) for m in
564.                 self.recent_metrics]
565.             return sum(values) / len(values) if values else 0.0
566.
567.         def rolling_velocity(self, attr: str) -> float:
568.             if len(self.recent_metrics) < 2:
569.                 return 0.0
570.             return (getattr(self.recent_metrics[-1], attr)
571.                 - getattr(self.recent_metrics[-2], attr))
572.
573.     @dataclass
574.     class SentinelRunContext:
575.         run_id: str
576.         timestamp: datetime
577.         contaminated_conditions: Optional[List[str]] = None
578.         last_regime: Optional[SystemRegime] = None

```

```

579.     def learning_allowed(self) -> bool:
580.         if not self.contaminated_conditions:
581.             return True
582.         return not any(c in LEARNING_EXCLUSIONS
583.                        for c in self.contaminated_conditions)
584.
585.
586. @dataclass
587. class RegimeProfile:
588.     scores: Dict[SystemRegime, float]
589.     dominant_regime: SystemRegime
590.     confidence: float
591.
592.
593. @dataclass
594. class SyntheticBorrower:
595.     borrower_id: str
596.     ach_segment: str
597.     delinquency_status: str
598.     repayment_shock: float
599.     confusion_tolerance: float
600.     emotional_state: float
601.     trust_state: float
602.     retry_propensity: float
603.     escalation_propensity: float
604.     abandonment_threshold: float
605.     prior_contact_count: int
606.     fraud_risk: float
607.     demographic_weight: float = 1.0
608.
609.
610. @dataclass
611. class CohortSummary:
612.     total_records: int = 0
613.     total_weighted_population: float = 0.0
614.     avg_repayment_shock: float = 0.5
615.     avg_confusion_tolerance: float = 0.5
616.     avg_escalation_propensity: float = 0.5
617.     avg_abandonment_threshold: float = 0.5
618.     max_repayment_shock: float = 0.0 # boundary marker
    - bimodal masking fix

```



```

619.         min_confusion_tolerance: float = 1.0    # boundary marker
        - bimodal masking fix
620.
621.
622. @dataclass
623. class TouchpointDelta:
624.     update_id: str
625.     channel: TouchpointChannel
626.     exposure_duration_secs: float
627.     friction_coefficient: float
628.     clarity_index: float
629.     randomize_verification: bool = False
630.
631.
632. #
        =====
633. # PART E — EVENT SYSTEM (Module 3)
634. #
        =====
635.
636. @dataclass(frozen=True)
637. class Event:
638.     event_type: str
639.     run_id: str
640.     timestamp: datetime
641.     payload: Dict[str, Any]
642.
643.
644. class EventStore:
645.     def __init__(self):
646.         self._events: List[Event] = []
647.
648.     def append(self, event: Event):
649.         self._events.append(event)
650.
651.     def all(self) -> List[Event]:
652.         return list(self._events)
653.
654.
655. class EventBus:
656.     def __init__(self, store: EventStore):

```

```

657.         self.store = store
658.
659.     def emit(self, event: Event):
660.         self.store.append(event)
661.
662.
663. #
=====
664. # PART F – DETERMINISTIC REPLAY ENGINE (Module 3)
665. #
=====
666.
667. class DeterministicReplay:
668.     def hash_event(self, event: Event) -> str:
669.         return hashlib.sha256(
670.             json.dumps(event.__dict__, default=str).encode()
671.         ).hexdigest()
672.
673.
674. class TraceReplayer:
675.     def __init__(self, store: EventStore):
676.         self.store = store
677.         self._replay = DeterministicReplay()
678.
679.     def rebuild(self) -> Dict[str, Any]:
680.         state: Dict[str, Any] = {"evaluations": [],
681. "advisories": [], "hashes": []}
682.         for e in self.store.all():
683.             state["hashes"].append(self._replay.hash_event(e))
684.             if e.event_type == "EVAL_RESULT":
685.                 state["evaluations"].append(e.payload)
686.             if e.event_type == "ADVISORY":
687.                 state["advisories"].append(e.payload)
688.         return state
689.
690. #
=====
691. # PART G – COHORT INGESTION PIPELINE (Module 2)

```

```

692.  #
        =====
693.
694.  class HighScaleCohortPipeline:
695.
696.      @staticmethod
697.      def parse_row(row: Dict[str, str], fallback_weight: float
        = 50.0) -> SyntheticBorrower:
698.          income      = float(row.get("Annual_Income", 50000))
699.          balance      = float(row.get("Loan_Balance", 0))
        # noqa: F841
700.          payment      = float(row.get("Monthly_Payment_Amount",
        0))
701.          amort         = float(row.get("Amortization_Ratio",
        100.0))
702.          recency       = row.get("Engagement_Recency", "Never")
703.          delinquency    = row.get("Delinquency_Status",
        "Current")
704.          tranche        = row.get("SAVE_Tranche", "T0")
705.          weight         = float(row.get("Demographic_Weight",
        fallback_weight))
706.
707.          tranche_mult    = 1.3 if tranche in ("T1", "T2",
        "T3") else 1.0
708.          repayment_shock = min(1.0, (payment * 12) /
        max(income, 1.0)
709.                                * tranche_mult * (amort /
        100.0))
710.
711.          delinquency_map = {"Current": 0.1, "1-29": 0.3,
        "30-59": 0.5,
712.                                "60-89": 0.7, "90+": 0.9}
713.          emotional_base  = delinquency_map.get(delinquency,
        0.2)
714.          emotional_state = min(1.0, emotional_base +
        repayment_shock * 0.3)
715.          trust_state      = max(0.0, 1.0 - repayment_shock *
        0.4 - emotional_base * 0.5)
716.
717.          recency_map      = {"<30d": 0.8, "1-12mo": 0.6,
        ">1yr": 0.4, "Never": 0.2}
718.          confusion_tolerance = recency_map.get(recency, 0.5)

```

```

719.
720.         retry_propensity      = min(1.0, 0.3 + emotional_state
    * 0.5
721.                                     + (0.2 if recency ==
    "<30d" else 0.0))
722.         escalation_propensity = min(1.0, (1.0 - trust_state)
    * 0.8
723.                                     + emotional_state * 0.2)
724.         abandonment_threshold = max(0.05, 1.0 -
    emotional_state * 0.6)
725.         prior_contacts        = {"<30d": 3, "1-12mo": 1,
    ">1yr": 2, "Never": 0}.get(recency, 0)
726.
727.         return SyntheticBorrower(
728.             borrower_id=row.get("Borrower_ID", "UNKNOWN"),
729.             ach_segment=tranche,
    delinquency_status=delinquency,
730.             repayment_shock=round(repayment_shock, 3),
731.             confusion_tolerance=round(confusion_tolerance,
    3),
732.             emotional_state=round(emotional_state, 3),
733.             trust_state=round(trust_state, 3),
734.             retry_propensity=round(retry_propensity, 3),
735.             escalation_propensity=round(escalation_propensity, 3),
736.             abandonment_threshold=round(abandonment_threshold, 3),
737.             prior_contact_count=prior_contacts,
738.             fraud_risk=0.0,
739.             demographic_weight=weight,
740.         )
741.
742.         def stream_cohort_file(self, file_path: str) ->
    Generator[SyntheticBorrower, None, None]:
743.             with open(file_path, mode="r", encoding="utf-8") as
    f:
744.                 for row in csv.DictReader(f):
745.                     yield self.parse_row(row)
746.
747.         def generate_summary(self, file_path: str) ->
    CohortSummary:

```

```

748.         count = w_sum = shock_acc = confusion_acc =
           escalation_acc = abandon_acc = 0.0
749.         max_shock, min_confusion = 0.0, 1.0
750.         n = 0
751.
752.         with open(file_path, mode="r", encoding="utf-8") as
           f:
753.             for row in csv.DictReader(f):
754.                 b = self.parse_row(row)
755.                 w = b.demographic_weight
756.                 n += 1
757.                 w_sum         += w
758.                 shock_acc     += b.repayment_shock         * w
759.                 confusion_acc += b.confusion_tolerance * w
760.                 escalation_acc += b.escalation_propensity * w
761.                 abandon_acc   += b.abandonment_threshold * w
762.                 max_shock     = max(max_shock,
           b.repayment_shock)
763.                 min_confusion = min(min_confusion,
           b.confusion_tolerance)
764.
765.             if w_sum == 0:
766.                 return CohortSummary()
767.
768.             return CohortSummary(
769.                 total_records=n, total_weighted_population=w_sum,
770.                 avg_repayment_shock=round(shock_acc / w_sum,
           3),
771.                 avg_confusion_tolerance=round(confusion_acc /
           w_sum, 3),
772.                 avg_escalation_propensity=round(escalation_acc /
           w_sum, 3),
773.                 avg_abandonment_threshold=round(abandon_acc /
           w_sum, 3),
774.                 max_repayment_shock=round(max_shock, 3),
775.                 min_confusion_tolerance=round(min_confusion, 3),
776.             )
777.
778.
779. #
           =====
780. # PART H – PSYCHOMETRIC TRANSPOSITION ENGINE (Module 2)

```

```

781. #
      =====
782.
783. class PsychometricTranspositionEngine:
784.
785.     def simulate_borrower_reaction(
786.         self, borrower: SyntheticBorrower, delta:
787.         TouchpointDelta
788.     ) -> SyntheticBorrower:
789.
790.         c_tol = borrower.confusion_tolerance
791.         emo    = borrower.emotional_state
792.         trust  = borrower.trust_state
793.
794.         if delta.clarity_index < 0.5:
795.             c_tol = max(0.0, c_tol - (0.5 -
796.             delta.clarity_index) * 0.5)
797.             emo    = min(1.0, emo    + (0.5 -
798.             delta.clarity_index) * 0.3)
799.         else:
800.             c_tol = min(1.0, c_tol + (delta.clarity_index -
801.             0.5) * 0.2)
802.             trust = min(1.0, trust + (delta.clarity_index -
803.             0.5) * 0.1)
804.
805.         if delta.friction_coefficient > 0.4:
806.             excess = delta.friction_coefficient - 0.4
807.             c_tol  = max(0.0, c_tol - excess * 0.4)
808.             emo    = min(1.0, emo    + excess * 0.4)
809.
810.         if (delta.channel in (TouchpointChannel.IVR,
811.             TouchpointChannel.WEB_PORTAL)
812.             and delta.exposure_duration_secs > 120.0
813.             and borrower.repayment_shock > 0.6):
814.             emo = min(1.0, emo + 0.10)
815.
816.         return SyntheticBorrower(
817.             borrower_id=borrower.borrower_id,
818.             ach_segment=borrower.ach_segment,
819.             delinquency_status=borrower.delinquency_status,
820.             repayment_shock=borrower.repayment_shock,
821.             confusion_tolerance=round(c_tol, 3),

```

```

816.         emotional_state=round(emo, 3),
817.         trust_state=round(trust, 3),
818.         retry_propensity=round(min(1.0,
    borrower.retry_propensity + (1.0 - c_tol) * 0.2), 3),
819.         escalation_propensity=round(min(1.0, (1.0 -
    trust) * 0.7 + emo * 0.3), 3),
820.         abandonment_threshold=round(max(0.05,
    borrower.abandonment_threshold - emo * 0.2), 3),
821.         prior_contact_count=borrower.prior_contact_count,
822.         fraud_risk=borrower.fraud_risk,
823.         demographic_weight=borrower.demographic_weight,
824.     )
825.
826.
827. #
    =====
828. # PART I - BEHAVIOR MODELS (Module 1: baseline + log-odds)
829. #
    =====
830.
831. class PredictiveReactionSandbox:
832.     """Nonlinear baseline behavior model
    (sigmoid-bounded)."""
833.
834.     def generate_projected_metrics(
835.         self, base: Metrics, summ: CohortSummary, d:
    TouchpointDelta
836.     ) -> Metrics:
837.         aht = self._project_aht(base.aht, d)
838.         abandonment = self._project_abandon_rate(base, summ,
    d)
839.         reentry = self._project_reentry_rate(summ)
840.         repeat_auth = self._project_repeat_auth_rate(summ)
841.         transfer = self._project_transfer_rate(summ)
842.         containment =
    self._project_containment(abandonment.p, transfer.p)
843.         callback = self._project_callback_rate(base,
    abandonment.unclipped)
844.         backoffice = self._project_backoffice_trigger(base,
    reentry.p)
845.         determinism = self._project_determinism_index(base,
    d)

```

```

846.         short_disc = self._project_short_disconnect(base,
summ)
847.
848.         return Metrics(
849.             containment=round(containment, 3),
850.             aht=round(aht, 1),
851.             callback_rate=round(callback.p, 3),
852.             abandon_rate=round(abandonment.p, 3),
853.             backoffice_trigger=round(backoffice, 3),
854.             determinism_index=round(determinism, 3),
855.             repeat_auth_rate=round(repeat_auth.p, 3),
856.             ivr_reentry_rate=round(reentry.p, 3),
857.             transfer_rate=round(transfer.p, 3),
858.             short_disconnect_rate=round(short_disc, 3),
859.             queue_depth=base.queue_depth,
860.
            unclipped_abandon_rate=round(abandonment.unclipped, 4),
861.             unclipped_callback_rate=round(callback.unclipped,
4),
862.
            unclipped_ivr_reentry_rate=round(reentry.unclipped, 4),
863.         )
864.
865.         def _project_aht(self, base_aht: float, d:
TouchpointDelta) -> float:
866.             inc = d.exposure_duration_secs
867.             if d.friction_coefficient > 0.5:
868.                 inc += d.friction_coefficient * 40
869.             return base_aht + inc
870.
871.         def _project_abandon_rate(self, base: Metrics, summ:
CohortSummary,
872.                                     d: TouchpointDelta) ->
RatePair:
873.             mult          = 1 + (1 - summ.avg_abandonment_threshold)
* 0.5
874.             unclipped = base.abandon_rate * mult * (1 +
d.friction_coefficient * 0.3)
875.             bounded    = soft_bound_sigmoid(5 * (unclipped - 0.5))
876.             return RatePair(unclipped=unclipped, p=bounded)
877.

```



```

878.         def _project_reentry_rate(self, summ: CohortSummary) ->
            RatePair:
879.             unclipped = (1 - summ.avg_confusion_tolerance) * 0.40
880.             return RatePair(unclipped=unclipped,
                p=soft_bound_sigmoid(5 * (unclipped - 0.5)))
881.
882.         def _project_repeat_auth_rate(self, summ: CohortSummary)
            -> RatePair:
883.             unclipped = (1 - summ.avg_confusion_tolerance) * 0.25
884.             return RatePair(unclipped=unclipped,
                p=soft_bound_sigmoid(5 * (unclipped - 0.5)))
885.
886.         def _project_transfer_rate(self, summ: CohortSummary) ->
            RatePair:
887.             unclipped = summ.avg_escalation_propensity * 0.85
888.             return RatePair(unclipped=unclipped,
                p=soft_bound_sigmoid(5 * (unclipped - 0.5)))
889.
890.         def _project_containment(self, p_abandon: float,
            p_transfer: float) -> float:
891.             return max(0.0, 1.0 - p_abandon - p_transfer)
892.
893.         def _project_callback_rate(self, base: Metrics,
            unclipped_abandon: float) -> RatePair:
894.             unclipped = base.callback_rate * (1.1 if
                unclipped_abandon > 0.15 else 1.0)
895.             return RatePair(unclipped=unclipped,
                p=soft_bound_sigmoid(5 * (unclipped - 0.5)))
896.
897.         def _project_backoffice_trigger(self, base: Metrics,
            p_reentry: float) -> float:
898.             return soft_bound_sigmoid(5 *
                ((base.backoffice_trigger + p_reentry * 0.15) - 0.5))
899.
900.         def _project_determinism_index(self, base: Metrics, d:
            TouchpointDelta) -> float:
901.             return max(0.01, base.determinism_index - 0.40) if
                d.randomize_verification \
902.                 else base.determinism_index
903.
904.         def _project_short_disconnect(self, base: Metrics, summ:
            CohortSummary) -> float:

```

```

905.         return base.short_disconnect_rate * (1 + (1 -
summ.avg_abandonment_threshold) * 0.5)
906.
907.
908. class PredictiveReactionSandboxLogOdds:
909.     """Log-odds behavior model – additive in log-odds space,
more stable curves."""
910.
911.     def generate_projected_metrics(
912.         self, base: Metrics, summ: CohortSummary, d:
TouchpointDelta
913.     ) -> Metrics:
914.         aht = self._aht(base, d)
915.         abandonment = self._abandon(base, summ, d)
916.         reentry = self._reentry(summ)
917.         repeat_auth = self._repeat_auth(summ)
918.         transfer = self._transfer(summ)
919.         containment = max(0.0, 1.0 - abandonment.p -
transfer.p)
920.         callback = self._callback(base, abandonment.p)
921.         backoffice = self._backoffice(base, reentry.p)
922.         determinism = self._determinism(base, d)
923.         short_disc = base.short_disconnect_rate * (1 + (1 -
summ.avg_abandonment_threshold) * 0.4)
924.
925.         return Metrics(
926.             containment=round(containment, 3),
927.             aht=round(aht, 1),
928.             callback_rate=round(callback.p, 3),
929.             abandon_rate=round(abandonment.p, 3),
930.             backoffice_trigger=round(backoffice, 3),
931.             determinism_index=round(determinism, 3),
932.             repeat_auth_rate=round(repeat_auth.p, 3),
933.             ivr_reentry_rate=round(reentry.p, 3),
934.             transfer_rate=round(transfer.p, 3),
935.             short_disconnect_rate=round(short_disc, 3),
936.             queue_depth=base.queue_depth,
937.
unclipped_abandon_rate=round(abandonment.unclipped, 4),
938.             unclipped_callback_rate=round(callback.unclipped,
4),

```

```

939.         unclipped_ivr_reentry_rate=round(reentry.unclipped, 4),
940.         )
941.
942.         def _aht(self, base: Metrics, d: TouchpointDelta) ->
float:
943.             return base.aht + d.exposure_duration_secs +
d.friction_coefficient * 25
944.
945.         def _abandon(self, base: Metrics, summ: CohortSummary, d:
TouchpointDelta) -> RatePair:
946.             adj = (1 - summ.avg_abandonment_threshold) * 1.2 +
d.friction_coefficient * 0.9
947.             p = inv_logit(logit(base.abandon_rate) + adj)
948.             return RatePair(unclipped=p, p=p)
949.
950.         def _reentry(self, summ: CohortSummary) -> RatePair:
951.             p = inv_logit(logit(0.10) + (1 -
summ.avg_confusion_tolerance) * 1.5)
952.             return RatePair(unclipped=p, p=p)
953.
954.         def _repeat_auth(self, summ: CohortSummary) -> RatePair:
955.             p = inv_logit(logit(0.05) + (1 -
summ.avg_confusion_tolerance) * 1.0)
956.             return RatePair(unclipped=p, p=p)
957.
958.         def _transfer(self, summ: CohortSummary) -> RatePair:
959.             p = inv_logit(logit(0.12) +
summ.avg_escalation_propensity * 1.3)
960.             return RatePair(unclipped=p, p=p)
961.
962.         def _callback(self, base: Metrics, p_abandon: float) ->
RatePair:
963.             p = inv_logit(logit(base.callback_rate) + (p_abandon
- base.abandon_rate) * 2.0)
964.             return RatePair(unclipped=p, p=p)
965.
966.         def _backoffice(self, base: Metrics, p_reentry: float) ->
float:
967.             return inv_logit(logit(base.backoffice_trigger) +
p_reentry * 0.5)
968.

```

```

969.         def _determinism(self, base: Metrics, d: TouchpointDelta)
          -> float:
970.             return max(0.01, base.determinism_index - 0.30) if
                d.randomize_verification \
971.                 else base.determinism_index
972.
973.
974. #
          =====
975. # PART J – LYAPUNOV STABILITY FRAMEWORK (Module 1)
976. #
          =====
977.
978. def set_lyapunov_weights(overrides: Optional[Dict[str,
            float]] = None) -> None:
979.     if not overrides:
980.         return
981.     for k, v in overrides.items():
982.         if k in LYAPUNOV_ENERGY_WEIGHTS and v > 0:
983.             LYAPUNOV_ENERGY_WEIGHTS[k] = float(v)
984.
985.
986. class LyapunovStabilityVerifier:
987.     """Quadratic Lyapunov energy:  $V = \sum w_i (\Delta x_i)^2$ """
988.
989.     @staticmethod
990.     def compute_energy_state_vectorized(
991.         b: Metrics, c: Metrics, return_breakdown: bool =
            False
992.     ):
993.         delta = np.array([
994.             c.abandon_rate - b.abandon_rate,
995.             c.ivr_reentry_rate - b.ivr_reentry_rate,
996.             c.transfer_rate - b.transfer_rate,
997.             (c.aht - b.aht) / (abs(b.aht) + 1e-9),
998.         ])
999.         w = np.array([
1000.             max(LYAPUNOV_ENERGY_WEIGHTS["abandon_rate"],
                1e-9),
1001.             max(LYAPUNOV_ENERGY_WEIGHTS["ivr_reentry_rate"],
                1e-9),

```

```

1002.                max(LYAPUNOV_ENERGY_WEIGHTS["transfer_rate"],
1003.                    1e-9),
1004.                max(LYAPUNOV_ENERGY_WEIGHTS["normalized_aht"],
1005.                    1e-9),
1006.            ])
1007.            terms = w * (delta ** 2)
1008.            energy = float(np.sum(terms))
1009.
1010.            if return_breakdown:
1011.                return energy, {
1012.                    "abandon_rate": float(terms[0]),
1013.                    "ivr_reentry_rate": float(terms[1]),
1014.                    "transfer_rate": float(terms[2]),
1015.                    "normalized_aht": float(terms[3]),
1016.                }
1017.            return energy
1018.
1019.        @staticmethod
1020.        def compute_energy_batch(baseline: Metrics, metrics_list:
1021.            List[Metrics]) -> List[float]:
1022.            return
1023.            [LyapunovStabilityVerifier.compute_energy_state_vectorized(baseline, m)
1024.             for m in metrics_list]
1025.
1026.        @staticmethod
1027.        def delta_energy(b: Metrics, c: Metrics) -> dict:
1028.            Vb =
1029.                LyapunovStabilityVerifier.compute_energy_state_vectorized(b, b)
1030.            Vc =
1031.                LyapunovStabilityVerifier.compute_energy_state_vectorized(b, c)
1032.            return {"V_baseline": Vb, "V_current": Vc, "delta_V":
1033.                Vc - Vb}
1034.
1035.        @staticmethod
1036.        def classify_stability(energy: float,
1037.                               stable_threshold: float = 1e-4,
1038.                               marginal_threshold: float = 1e-2)
1039.            -> str:
1040.            if energy <= stable_threshold:
1041.                return "stable"
1042.            if energy <= marginal_threshold:

```

```

1035.         return "marginal"
1036.     return "unstable"
1037.
1038.     @staticmethod
1039.     def auto_tune_weights(
1040.         baseline: Metrics,
1041.         training_data: List[Metrics],
1042.         weight_grid: Tuple[float, ...] = (0.1, 1.0, 2.0, 5.0)
1043.     ) -> dict:
1044.         best_weights, best_score = None, float("inf")
1045.         for wa in weight_grid:
1046.             for wr in weight_grid:
1047.                 for wt in weight_grid:
1048.                     for wn in weight_grid:
1049.                         LYAPUNOV_ENERGY_WEIGHTS.update({
1050.                             "abandon_rate": wa,
1051.                             "ivrr_reentry_rate": wr,
1052.                             "transfer_rate": wt,
1053.                             "normalized_aht": wn,
1054.                         })
1055.                         score = sum(
1056.                             LyapunovStabilityVerifier.compute_energy_state_vectorized(basel
1057. ine, m)
1058.                             for m in training_data
1059.                         )
1060.                         if score < best_score:
1061.                             best_score, best_weights = score,
1062.                             (wa, wr, wt, wn)
1063.
1064.         if best_weights:
1065.             wa, wr, wt, wn = best_weights
1066.             LYAPUNOV_ENERGY_WEIGHTS.update({
1067.                 "abandon_rate": wa, "ivrr_reentry_rate": wr,
1068.                 "transfer_rate": wt, "normalized_aht": wn,
1069.             })
1070.         return {"best_weights": best_weights, "best_score":
1071.             best_score}
1072.
1073. #
1074. =====

```



```

1099.             if ab_v > 0:
1100.                 scores[SystemRegime.PANIC] +=
min(VELOCITY_MAX_BOUND, ab_v * 3.0)
1101.
1102.             max_score = max(scores.values())
1103.             if max_score <= 0:
1104.                 dominant, confidence = SystemRegime.STABLE, 1.0
1105.             else:
1106.                 normalized = {r: round(s / max_score, 3) for r, s
in scores.items()}
1107.                 dominant = max(normalized, key=normalized.get)
1108.                 confidence = normalized[dominant]
1109.                 scores = normalized
1110.
1111.             if previous_regime:
1112.                 prev_s = scores.get(previous_regime, 0.0)
1113.                 dom_s = scores.get(dominant, 0.0)
1114.                 if dominant != previous_regime and dom_s - prev_s
< HYSTERESIS_MARGIN:
1115.                     dominant, confidence = previous_regime,
prev_s
1116.
1117.             return RegimeProfile(scores=scores,
dominant_regime=dominant,
1118.                                     confidence=round(confidence, 3))
1119.
1120.
1121. #
=====
1122. # PART L - GOVERNANCE ENGINE
1123. #
=====
1124.
1125. class GovernanceEngine:
1126.
1127.     def evaluate_fraud(
1128.         self, baseline: Metrics, post: Metrics, run_id: str,
ts: datetime,
1129.         profile: RegimeProfile
1130.     ) -> PopulationResult:
1131.         delta = post.determinism_index -
baseline.determinism_index

```



```

1132.         result = (SentinelResult.RISK_INCREASING if delta >
1133.                     FRAUD_DETERMINISM_THRESHOLD
1134.                     else SentinelResult.NEUTRAL)
1135.         return PopulationResult(
1136.             population=PopulationType.FRAUD, result=result,
1137.             rationale=f"fraud_delta={delta:.3f}",
1138.             dominant_regime=profile.dominant_regime,
1139.             regime_confidence=profile.confidence,
1140.             regime_scores={k.value: v for k, v in
1141.                             profile.scores.items()}),
1142.             run_id=run_id, timestamp=ts,
1143.         )
1144.
1145.     def evaluate_borrower(
1146.         self,
1147.         pop: PopulationType,
1148.         baseline: Metrics,
1149.         post: Metrics,
1150.         priorities: List[str],
1151.         profile: RegimeProfile,
1152.         run_id: str,
1153.         ts: datetime,
1154.         cohort_summary: Optional[CohortSummary] = None,
1155.     ) -> PopulationResult:
1156.
1157.         regime = profile.dominant_regime
1158.         tolerances = REGIME_TOLERANCE_MATRICES.get(regime,
1159.             REGIME_TOLERANCE_MATRICES[SystemRegime.STABLE])
1160.
1161.         # INTERLOCK: adversarial regime hijacking - if
1162.         # determinism rises, lock to FRAUD_WAVE
1163.         if post.determinism_index >
1164.             baseline.determinism_index:
1165.             tolerances =
1166.                 REGIME_TOLERANCE_MATRICES[SystemRegime.FRAUD_WAVE]
1167.
1168.         shock_modifier = 1.0
1169.         tolerance_modifier = 1.0
1170.
1171.         # ANTI-AGGREGATION FILTER: bimodal masking fix
1172.         (Simpson's Paradox guard)

```

```

1167.         if cohort_summary:
1168.             blended_shock      =
1169.                 (cohort_summary.avg_repayment_shock
1170.                 +
1171.                 cohort_summary.max_repayment_shock) / 2.0
1172.             blended_tolerance =
1173.                 (cohort_summary.avg_confusion_tolerance
1174.                 +
1175.                 cohort_summary.min_confusion_tolerance) / 2.0
1176.             shock_modifier      = 1.0 + blended_shock * 0.5
1177.             tolerance_modifier = max(0.5, blended_tolerance)
1178.
1179.             checks = {
1180.                 "operationalefficiency": (
1181.                     (post.aht - baseline.aht) <=
1182.                     tolerances["max_aht_delta"] * shock_modifier
1183.                     and post.callback_rate <=
1184.                     baseline.callback_rate + tolerances["max_callback_delta"]
1185.                 ),
1186.                 "futureselfresolution": (
1187.                     post.backoffice_trigger <=
1188.                     baseline.backoffice_trigger
1189.                     + BACKOFFICE_REGRESSION_THRESHOLD
1190.                 ),
1191.                 "informationaccuracy": (
1192.                     post.abandon_rate <= baseline.abandon_rate
1193.                     + tolerances["max_abandon_delta"] *
1194.                     tolerance_modifier
1195.                 ),
1196.                 "borrowerpsychologicalsafety": (
1197.                     post.containment >= baseline.containment
1198.                     + tolerances["min_containment_delta"] /
1199.                     tolerance_modifier
1200.                 ),
1201.                 "queuepreservation": (
1202.                     post.queue_depth <
1203.                     tolerances["max_queue_depth"]
1204.                 ),
1205.                 "determinismreduction": (
1206.                     post.determinism_index <=
1207.                     baseline.determinism_index
1208.                 ),
1209.             }

```

```

1198.         }
1199.
1200.         failures = [p for p in priorities if not
        checks.get(p, True)]
1201.         result = SentinelResult.REGRESSIVE if failures else
        SentinelResult.NEUTRAL
1202.         rationale = (f"Failed priorities: {'',
        '.join(failures)}"
1203.                     if failures else "No priority regression
        detected")
1204.
1205.         return PopulationResult(
1206.             population=pop, result=result,
        rationale=rationale,
1207.             dominant_regime=profile.dominant_regime,
1208.             regime_confidence=profile.confidence,
1209.             regime_scores={k.value: v for k, v in
        profile.scores.items()}),
1210.         run_id=run_id, timestamp=ts,
1211.     )
1212.
1213.
1214. #
        =====
1215. # PART M – SENTINEL CORE ORCHESTRATOR
1216. #
        =====
1217.
1218. class SentinelIVR:
1219.
1220.     def __init__(
1221.         self,
1222.         context: SentinelRunContext,
1223.         bus: EventBus,
1224.         temporal_window: Optional[TemporalWindow] = None,
1225.     ):
1226.         self.context = context
1227.         self.bus = bus
1228.         self.temporal_window = temporal_window
1229.         self.regime_detector = RegimeDetector()
1230.         self.governance = GovernanceEngine()

```

```

1231.         self.transposition      =
1232.             PsychometricTranspositionEngine()
1233.         self.sandbox_baseline = PredictiveReactionSandbox()
1234.         self.sandbox_logodds   =
1235.             PredictiveReactionSandboxLogOdds()
1236.
1237.         #
1238.         -----
1239.         # HIGH-SCALE UPDATE (streaming cohort path)
1240.         #
1241.         -----
1242.
1243.     def simulate_high_scale_update(
1244.         self,
1245.         population: PopulationType,
1246.         baseline_metrics: Metrics,
1247.         cohort_generator: Generator[SyntheticBorrower, None,
1248.             None],
1249.         proposed_update: TouchpointDelta,
1250.         sandbox: str = "log_odds",
1251.     ) -> PopulationResult:
1252.
1253.         count = w_sum = shock_acc = confusion_acc =
1254.             escalation_acc = abandon_acc = 0.0
1255.         max_shock, min_confusion = 0.0, 1.0
1256.
1257.         for borrower in cohort_generator:
1258.             mutated =
1259.                 self.transposition.simulate_borrower_reaction(borrower,
1260.                     proposed_update)
1261.
1262.             w          = mutated.demographic_weight
1263.             count      += 1
1264.             w_sum      += w
1265.             shock_acc  += mutated.repayment_shock * w
1266.             confusion_acc += mutated.confusion_tolerance * w
1267.             escalation_acc += mutated.escalation_propensity *
1268.                 w
1269.             abandon_acc += mutated.abandonment_threshold *
1270.                 w
1271.
1272.             max_shock = max(max_shock,
1273.                 mutated.repayment_shock)

```

```

1261.         min_confusion = min(min_confusion,
1262.                                mutated.confusion_tolerance)
1263.         if w_sum == 0:
1264.             w_sum = 1.0
1265.
1266.         mutated_summary = CohortSummary(
1267.             total_records=int(count),
1268.             total_weighted_population=w_sum,
1269.             avg_repayment_shock=shock_acc / w_sum,
1270.             avg_confusion_tolerance=confusion_acc / w_sum,
1271.             avg_escalation_propensity=escalation_acc / w_sum,
1272.             avg_abandonment_threshold=abandon_acc / w_sum,
1273.             max_repayment_shock=max_shock,
1274.             min_confusion_tolerance=min_confusion,
1275.         )
1276.         model = (self.sandbox_logodds if sandbox ==
1277.                  "log_odds"
1278.                  else self.sandbox_baseline)
1279.         post =
1280.             model.generate_projected_metrics(baseline_metrics,
1281.                                               mutated_summary,
1282.                                               proposed_update)
1283.         return self.evaluate(population, baseline_metrics,
1284.                               post, mutated_summary)
1285.
1286.         #
1287.         -----
1288.         # MAIN EVALUATION
1289.         #
1290.         -----
1291.
1292.         def evaluate(
1293.             self,
1294.             population: PopulationType,
1295.             baseline: Metrics,
1296.             post: Metrics,
1297.             cohort_summary: Optional[CohortSummary] = None,
1298.         ) -> PopulationResult:

```

```

1294.         self.bus.emit(Event(
1295.             event_type="EVAL_START",
1296.             run_id=self.context.run_id,
1297.             timestamp=self.context.timestamp,
1298.             payload={"population": population.value},
1299.         ))
1300.         profile = self.regime_detector.detect(post,
1301.             self.temporal_window,
1302.             self.context.last_regime)
1303.         self.context.last_regime = profile.dominant_regime
1304.         priorities =
1305.             PRIORITY_PROFILES.get(profile.dominant_regime.value,
1306.             PRIORITY_PROFILES["stable"])
1307.         if population == PopulationType.FRAUD:
1308.             result = self.governance.evaluate_fraud(
1309.                 baseline, post, self.context.run_id,
1310.                 self.context.timestamp, profile)
1311.             elif not self.context.learning_allowed():
1312.                 result = PopulationResult(
1313.                     population=population,
1314.                     result=SentinelResult.NEUTRAL,
1315.                     rationale="Learning suspended – contaminated
1316.                     conditions",
1317.                     dominant_regime=profile.dominant_regime,
1318.                     regime_confidence=profile.confidence,
1319.                     regime_scores={k.value: v for k, v in
1320.                         profile.scores.items()}),
1321.                     run_id=self.context.run_id,
1322.                     timestamp=self.context.timestamp,
1323.                 )
1324.         else:
1325.             result = self.governance.evaluate_borrower(
1326.                 population, baseline, post, priorities,
1327.                 profile,
1328.                 self.context.run_id, self.context.timestamp,
1329.                 cohort_summary,
1330.             )

```

```

1324.         self.bus.emit(Event(
1325.             event_type="EVAL_RESULT",
1326.             run_id=self.context.run_id,
1327.             timestamp=self.context.timestamp,
1328.             payload={"population": population.value,
1329.                     "result": result.result.value,
1330.                     "rationale": result.rationale},
1331.             ))
1332.         return result
1333.
1334.     #
1335.     -----
1336.     # ADVISORY
1337.     #
1338.     -----
1339.     def advise(self, profile: RegimeProfile) -> Dict[str,
1340.         Any]:
1341.         dominant = profile.dominant_regime
1342.         directives = {
1343.             SystemRegime.PANIC:      ("PANIC",      ["minimize
1344.                 cognitive load",
1345.                 "prioritize stabilization messaging",
1346.                 "authoritative repayment clarification",
1347.                 "protect
1348.                 queue"]),
1349.             SystemRegime.CONFUSION:  ("CONFUSION",  ["reduce
1350.                 recursive IVR traversal",
1351.                 "increase
1352.                 intent disambiguation",
1353.                 "accelerate borrower routing"]),
1354.             SystemRegime.SATURATED:  ("SATURATED",  ["protect
1355.                 queue integrity",
1356.                 "defer
1357.                 noncritical workflows",
1358.                 "shorten
1359.                 branch depth"]),

```

```

1351.         SystemRegime.SURGE:         ("SURGE",
1352.         ["prioritize informational consistency",
                                     "borrower
         stabilization over efficiency"])),
1353.         SystemRegime.FRAUD_WAVE: ("FRAUD_WAVE", ["reduce
         deterministic repeatability",
1354.                                     "increase
         challenge variation"])),
1355.     }
1356.     profile_name, dirs = directives.get(dominant,
1357.                                         ("DEFAULT",
         ["standard optimization conditions"]))
1358.     advice = {"profile": profile_name, "directives":
         dirs}
1359.     self.bus.emit(Event(
1360.         event_type="ADVISORY",
         run_id=self.context.run_id,
1361.         timestamp=self.context.timestamp, payload=advice,
1362.     ))
1363.     return advice
1364.
1365.
1366. #
         =====
1367. # PART N – WORKFORCE STABILITY MONITOR   (Module 1)
1368. #
         =====
1369.
1370. class WorkforceStabilityMonitor:
1371.
1372.     def __init__(self, baseline: Metrics):
1373.         self.baseline = baseline
1374.
1375.     def assess_future_stability(
1376.         self,
1377.         behavior_model,           # PredictiveReactionSandbox
         or LogOdds
1378.         horizon: int,
1379.         summ: CohortSummary,
1380.         delta_schedule: Callable[[int], TouchpointDelta],
1381.         stable_threshold: float = 1e-4,
1382.         marginal_threshold: float = 1e-2,

```



```

1383.         ) -> dict:
1384.             predictions: List[Metrics] = []
1385.             for t in range(horizon):
1386.                 d = delta_schedule(t)
1387.                 m =
1388.                     behavior_model.generate_projected_metrics(self.baseline, summ,
1389.                         d)
1390.                 predictions.append(m)
1391.             energies =
1392.                 LyapunovStabilityVerifier.compute_energy_batch(self.baseline,
1393.                     predictions)
1394.             classifications =
1395.                 [LyapunovStabilityVerifier.classify_stability(e,
1396.                     stable_threshold,
1397.                     marginal_threshold)
1398.                     for e in energies]
1399.             return {
1400.                 "predictions": predictions,
1401.                 "energies": energies,
1402.                 "delta_v": energies,
1403.                 "stability_classification": classifications,
1404.             }
1405.
1406. #
1407.
1408. # PART O – STABILITY DASHBOARD (Module 1)
1409. #
1410.
1411.
1412.
1413.
1414.
1415.
1416.
1417.
1418.
1419.
1420.
1421.
1422.
1423.
1424.
1425.
1426.
1427.
1428.
1429.
1430.
1431.
1432.
1433.
1434.
1435.
1436.
1437.
1438.
1439.
1440.
1441.
1442.
1443.
1444.
1445.
1446.
1447.
1448.
1449.
1450.
1451.
1452.
1453.
1454.
1455.
1456.
1457.
1458.
1459.
1460.
1461.
1462.
1463.
1464.
1465.
1466.
1467.
1468.
1469.
1470.
1471.
1472.
1473.
1474.
1475.
1476.
1477.
1478.
1479.
1480.
1481.
1482.
1483.
1484.
1485.
1486.
1487.
1488.
1489.
1490.
1491.
1492.
1493.
1494.
1495.
1496.
1497.
1498.
1499.
1500.

```

```

1415.         plt.legend()
1416.         plt.tight_layout()
1417.         plt.savefig(filename)
1418.         plt.close()
1419.
1420.     @staticmethod
1421.     def plot_classification_timeline(classifications:
        List[str],
1422.                                     filename: str =
        "classification_timeline.png"):
1423.         mapping = {"stable": 0, "marginal": 1, "unstable": 2}
1424.         y = [mapping.get(c, 1) for c in classifications]
1425.         plt.figure(figsize=(10, 4))
1426.         plt.step(range(len(y)), y, where="mid")
1427.         plt.yticks([0, 1, 2], ["stable", "marginal",
        "unstable"])
1428.         plt.title("Stability Classification Timeline")
1429.         plt.xlabel("Time Step")
1430.         plt.ylabel("Classification")
1431.         plt.tight_layout()
1432.         plt.savefig(filename)
1433.         plt.close()
1434.
1435.     @staticmethod
1436.     def build_report(energies: List[float], classifications:
        List[str],
1437.                     filename: str = "stability_report.txt")
        -> str:
1438.         report = "Stability Analytics Report\n" + "=" * 40 +
        "\n"
1439.         for i, (e, c) in enumerate(zip(energies,
        classifications)):
1440.             report += f"Step {i}: Energy={e:.6f},
        State={c}\n"
1441.         with open(filename, "w") as f:
1442.             f.write(report)
1443.         return filename
1444.
1445.
1446. #
        =====
1447. # PART P – BENCHMARK HARNESS (Module 1)

```

```

1448. #
=====
1449.
1450. class BenchmarkHarness:
1451.     """Compare baseline vs log-odds sandbox on identical
        inputs."""
1452.
1453.     FIELDS = ["aht", "abandon_rate", "callback_rate",
        "containment",
1454.               "transfer_rate", "ivr_reentry_rate",
        "repeat_auth_rate",
1455.               "backoffice_trigger", "short_disconnect_rate",
        "determinism_index"]
1456.
1457.     def __init__(self):
1458.         self.old = PredictiveReactionSandbox()
1459.         self.new = PredictiveReactionSandboxLogOdds()
1460.
1461.     def compare(self, base: Metrics, summ: CohortSummary,
1462.                 d: TouchpointDelta) -> List[str]:
1463.         old_m = self.old.generate_projected_metrics(base,
            summ, d)
1464.         new_m = self.new.generate_projected_metrics(base,
            summ, d)
1465.         lines = ["=== BENCHMARK: baseline vs log_odds ==="]
1466.         for f in self.FIELDS:
1467.             ov, nv = getattr(old_m, f, None), getattr(new_m,
            f, None)
1468.             if isinstance(ov, (int, float)) and
                isinstance(nv, (int, float)):
1469.                 lines.append(f"{f:28s} | baseline={ov:.4f} |
                    log_odds={nv:.4f}")
1470.             else:
1471.                 lines.append(f"{f:28s} | baseline={ov} |
                    log_odds={nv}")
1472.         return lines
1473.
1474.
1475. #
=====
1476. # PART Q – RL WORLD MODEL (Module 4)

```

```

1477. #
=====
1478.
1479. class WorldModel:
1480.     def __init__(self, dim: int = 8):
1481.         self.dim = dim
1482.         self.ws = [random.uniform(-0.02, 0.02) for _ in
range(dim)]
1483.         self.wa = [random.uniform(-0.02, 0.02) for _ in
range(dim)]
1484.
1485.     def encode(self, kpi: float) -> List[float]:
1486.         norm = (kpi - 100.0) / 100.0
1487.         return [math.tanh(norm * w) for w in self.ws]
1488.
1489.     def predict(self, kpi: float, delta: float) -> float:
1490.         z = self.encode(kpi)
1491.         pred = sum((zi + (delta / 120.0) * wi) for zi, wi in
zip(z, self.wa))
1492.         return math.tanh(pred) * 25.0
1493.
1494.     def update(self, kpi: float, delta: float, next_kpi:
float, lr: float) -> float:
1495.         if lr <= 0:
1496.             return 0.0
1497.         pred = self.predict(kpi, delta)
1498.         err = max(-10.0, min(10.0, next_kpi - pred))
1499.         for i in range(self.dim):
1500.             self.ws[i] = 0.98 * self.ws[i] + lr * err * 0.0008
1501.             self.wa[i] = 0.98 * self.wa[i] + lr * err * delta
* 0.00005
1502.         return abs(err)
1503.
1504.
1505. #
=====
1506. # PART R – POLICY + CRITIC (Module 4)
1507. #
=====
1508.
1509. class Policy:
1510.     def __init__(self, dim: int, n_agents: int):

```

```

1511.         self.dim          = dim
1512.         self.n_agents = n_agents
1513.         self.memory      = deque(maxlen=12)
1514.         self.lr          = 0.02
1515.         self.weights     = [[random.uniform(-0.01, 0.01) for _
    in range(n_agents)]
1516.                             for _ in range(dim)]
1517.
1518.     def select(self, z: List[float], beta: float):
1519.         attended = list(z)
1520.         for m in self.memory:
1521.             for i in range(self.dim):
1522.                 attended[i] += 0.05 * m[i]
1523.         self.memory.append(list(z))
1524.
1525.         logits = [sum(attended[i] * self.weights[i][k] for i
    in range(self.dim))
1526.                   for k in range(self.n_agents)]
1527.         m_val  = max(logits)
1528.         exps   = [math.exp(max(-20, 1 - m_val)) for l in
    logits]
1529.         s      = sum(exps) + 1e-9
1530.         probs  = [e / s for e in exps]
1531.
1532.         if beta <= 0:
1533.             return probs.index(max(probs)), attended, probs
1534.
1535.         r, c = random.random(), 0.0
1536.         for i, p in enumerate(probs):
1537.             c += p
1538.             if r <= c:
1539.                 return i, attended, probs
1540.         return self.n_agents - 1, attended, probs
1541.
1542.     def update(self, att: List[float], probs: List[float],
1543.                idx: int, adv: float, lr_mod: float = 1.0):
1544.         adv = max(-3.0, min(3.0, adv))
1545.         for k in range(self.n_agents):
1546.             grad = (1.0 if k == idx else 0.0) - probs[k]
1547.             for i in range(self.dim):

```

```

1548.                 self.weights[i][k] += self.lr * lr_mod * grad *
adv * att[i]
1549.
1550.     def reset(self):
1551.         self.memory.clear()
1552.
1553.
1554. class Critic:
1555.     def __init__(self, dim: int, hidden: int = 16, lr: float
= 0.005):
1556.         self.dim      = dim
1557.         self.hidden = hidden
1558.         self.lr       = lr
1559.         self.w1 = [[random.uniform(-0.1, 0.1) for _ in
range(hidden)] for _ in range(dim)]
1560.         self.b1 = [0.0] * hidden
1561.         self.w2 = [random.uniform(-0.1, 0.1) for _ in
range(hidden)]
1562.         self.b2 = 0.0
1563.
1564.     def value(self, z: List[float]):
1565.         h = [max(0.0, self.b1[j] + sum(z[i] * self.w1[i][j]
for i in range(self.dim)))
for j in range(self.hidden)]
1566.         v = self.b2 + sum(h[j] * self.w2[j] for j in
range(self.hidden))
1567.         return v, h
1568.
1569.
1570.     def update(self, z: List[float], td_target: float) ->
float:
1571.         v, h = self.value(z)
1572.         err = td_target - v
1573.         for j in range(self.hidden):
1574.             self.w2[j] += self.lr * err * h[j]
1575.         self.b2 += self.lr * err
1576.         return err
1577.
1578.
1579. #
=====
1580. # PART 5 – RL AGENTS (Module 4)

```

```

1581. #
=====
1582.
1583. class ConservativeAgent:
1584.     def tick(self, kpi: float, target: float) -> dict:
1585.         return {"delta": (target - kpi) * 0.05}
1586.
1587. class AggressiveAgent:
1588.     def tick(self, kpi: float, target: float) -> dict:
1589.         return {"delta": (target - kpi) * 0.30}
1590.
1591. class ReactiveAgent:
1592.     def tick(self, kpi: float, target: float) -> dict:
1593.         err = target - kpi
1594.         gain = 0.15 if abs(err) > 15 else 0.08
1595.         return {"delta": err * gain}
1596.
1597.
1598. #
=====
1599. # PART T – MANDATE LAYER (governance-constrained RL step)
1600. #
=====
1601.
1602. class MandateLayer:
1603.     """Hard governance bounds on RL agent proposals."""
1604.
1605.     @staticmethod
1606.     def enforce(delta: float, kpi: float, target: float,
1607.                 volatility: float) -> float:
1608.         dist = target - kpi
1609.         max_step = max(1.0, min(20.0, abs(dist) * 0.25))
1610.         max_step *= 1.0 / (1.0 + volatility)
1611.         delta = max(-max_step, min(max_step, delta))
1612.         proposed = kpi + delta
1613.         lower, upper = target - 60, target + 20
1614.         if proposed > upper:
1615.             delta = upper - kpi
1616.         elif proposed < lower:
1617.             delta = lower - kpi
1618.         return delta

```

```

1618.
1619.
1620. #
    =====
1621. # PART U – SOVEREIGN ENGINE (RL + governance loop) (Module
    7)
1622. #
    =====
1623.
1624. class IntegrityLayer:
1625.     def __init__(self):
1626.         self.err_history = deque(maxlen=10)
1627.
1628.     def analyze(self, world_err: float) -> float:
1629.         self.err_history.append(abs(world_err))
1630.         if len(self.err_history) < 3:
1631.             return 0.0
1632.         mean = sum(self.err_history) / len(self.err_history)
1633.         var = sum((x - mean) ** 2 for x in self.err_history)
            / len(self.err_history)
1634.         return max(0.0, min(1.0, math.sqrt(var) * 0.05))
1635.
1636.
1637. class SovereignEngine:
1638.     """RL training loop with regime-aware learning rates and
        mandate enforcement."""
1639.
1640.     REGIME_LR = {
1641.         "NORMAL": (1.0, 0.05),
1642.         "UNSTABLE": (0.7, 0.03),
1643.         "DEGRADED": (0.3, 0.01),
1644.         "RECOVERING": (0.9, 0.02),
1645.     }
1646.
1647.     def __init__(self, agents: list, dim: int = 8, gamma:
        float = 0.95,
1648.                 noise_scale: float = 3.0):
1649.         self.agents = agents
1650.         self.world = WorldModel(dim)
1651.         self.policy = Policy(dim, len(agents))
1652.         self.critic = Critic(dim)
1653.         self.integrity = IntegrityLayer()

```



```

1654.         self.gamma          = gamma
1655.         self.noise_scale = noise_scale
1656.         self.regime         = "NORMAL"
1657.         self.risk_hist      = deque(maxlen=6)
1658.         self.degraded_time = 0
1659.
1660.     def _update_regime(self, risk: float):
1661.         self.risk_hist.append(risk)
1662.         avg = sum(self.risk_hist) / len(self.risk_hist)
1663.         if avg > 0.7:
1664.             self.regime = "DEGRADED"
1665.         elif avg > 0.4:
1666.             self.regime = "UNSTABLE"
1667.         elif avg < 0.25:
1668.             self.regime = "RECOVERING"
1669.         else:
1670.             self.regime = "NORMAL"
1671.         self.degraded_time = max(0, self.degraded_time + (1
            if self.regime == "DEGRADED" else -1))
1672.
1673.     def run_cycle(self, start_kpi: float = 60.0, target:
        float = 100.0,
1674.                   steps: int = 60, target_shift_at: int = 30,
1675.                   target_shifted: float = 140.0) -> float:
1676.         kpi          = start_kpi
1677.         traj         = []
1678.         world_err     = 0.0
1679.         self.policy.reset()
1680.
1681.         for t in range(steps):
1682.             if t == target_shift_at:
1683.                 target = target_shifted
1684.
1685.             risk = self.integrity.analyze(world_err)
1686.             self._update_regime(risk)
1687.             lr_mod, beta = self.REGIME_LR[self.regime]
1688.
1689.             z          = self.world.encode(kpi)
1690.             idx, att, probs = self.policy.select(z, beta)
1691.             action       = self.agents[idx].tick(kpi, target)

```

```

1692.         volatility = min(2.0, world_err / 5.0) if
           world_err > 0 else 0.0
1693.         delta      =
           MandateLayer.enforce(action["delta"], kpi, target, volatility)
1694.
1695.         noise        = random.uniform(-self.noise_scale,
           self.noise_scale)
1696.         next_kpi     = max(0.0, (kpi + delta + noise) *
           0.995)
1697.         world_err    = self.world.update(kpi, delta,
           next_kpi, 0.02 * lr_mod)
1698.
1699.         traj.append((att, probs, idx, -abs(target -
           next_kpi)))
1700.         kpi = next_kpi
1701.
1702.         # Advantage normalization
1703.         rewards, g = [], 0.0
1704.         for _, _, _, r in reversed(traj):
1705.             g = r + self.gamma * g
1706.             rewards.append(g)
1707.         rewards.reverse()
1708.         mu = sum(rewards) / len(rewards)
1709.         std = math.sqrt(sum((x - mu) ** 2 for x in rewards) /
           len(rewards)) + 1e-6
1710.
1711.         for i, (att, probs, idx, _) in enumerate(traj):
1712.             adv = (rewards[i] - mu) / std
1713.             v, _ = self.critic.value(att)
1714.             self.critic.update(att, rewards[i])
1715.             self.policy.update(att, probs, idx, adv)
1716.
1717.         return kpi
1718.
1719.
1720. #
           =====
1721. # PART V – DUAL SYSTEM HARNESS   (Module 5 / 6)
1722. #
           =====
1723.
1724. @dataclass

```

```

1725. class StepResult:
1726.     score_a: float
1727.     score_b: float
1728.     action_a: str
1729.     action_b: str
1730.     energy_a: float
1731.     energy_b: float
1732.     coherence_a: float
1733.     coherence_b: float
1734.
1735.
1736. class DualSystemHarness:
1737.     """
1738.     Runs two Sentinel instances (A = current, B = proposed)
1739.     side-by-side
1740.     under independent noise, logs divergence, and surfaces a
1741.     governance verdict.
1742.     """
1743.     def __init__(self, system_a: SentinelIVR, system_b:
1744.         SentinelIVR,
1745.         noise: float = 0.0):
1746.         self.A = system_a
1747.         self.B = system_b
1748.         self.noise = noise
1749.         self.history: List[StepResult] = []
1750.         logger.info("DualSystemHarness initialized")
1751.
1752.         #
1753.         -----
1754.         # PERTURBATION
1755.         #
1756.         -----
1757.
1758.     def _perturb(self, metrics: Metrics) -> Metrics:
1759.         if self.noise <= 0:
1760.             return metrics
1761.         m = copy.deepcopy(metrics)
1762.         m.abandon_rate = max(0.0, m.abandon_rate +
1763.             self.noise * (0.5 - (time.time() % 1)))
1764.         m.callback_rate = max(0.0, m.callback_rate +
1765.             self.noise * 0.01)

```

```

1760.         return m
1761.
1762.     #
-----
1763.     # SINGLE STEP
1764.     #
-----
1765.
1766.     def step(
1767.         self,
1768.         population: PopulationType,
1769.         baseline: Metrics,
1770.         post: Metrics,
1771.         cohort: Optional[CohortSummary] = None,
1772.     ) -> Tuple[PopulationResult, PopulationResult,
    StepResult]:
1773.
1774.         post_a = self._perturb(post)
1775.         post_b = self._perturb(post)
1776.
1777.         result_a = self.A.evaluate(population, baseline,
    post_a, cohort)
1778.         result_b = self.B.evaluate(population, baseline,
    post_b, cohort)
1779.
1780.         score_map = {SentinelResult.NEUTRAL: 0.0,
1781.                      SentinelResult.REGRESSIVE: 1.0,
1782.                      SentinelResult.RISK_INCREASING: 2.0}
1783.
1784.         step = StepResult(
1785.             score_a=score_map[result_a.result],
1786.             score_b=score_map[result_b.result],
1787.             action_a=result_a.result.value,
1788.             action_b=result_b.result.value,
1789.             energy_a=result_a.regime_confidence,
1790.             energy_b=result_b.regime_confidence,
1791.             coherence_a=float(result_a.dominant_regime ==
    result_b.dominant_regime),
1792.             coherence_b=float(result_b.dominant_regime ==
    result_a.dominant_regime),
1793.         )
1794.

```

```

1795.         self.history.append(step)
1796.
1797.         logger.info(
1798.             f"step | score_delta={abs(step.score_a -
1799.                 step.score_b):.4f} | "
1800.             f"action_div={int(step.action_a !=
1801.                 step.action_b)} | "
1802.             f"regime_coherence={step.coherence_a:.1f} | "
1803.             f"regime_A={result_a.dominant_regime.value} | "
1804.             f"regime_B={result_b.dominant_regime.value}"
1805.         )
1806.
1807.         return result_a, result_b, step
1808.
1809.         #
1810.         -----
1811.         # VERDICT – governance handshake
1812.         #
1813.         -----
1814.
1815.         def verdict(self) -> Dict[str, Any]:
1816.             if not self.history:
1817.                 return {"verdict": "NO_DATA"}
1818.
1819.             regressions = sum(1 for h in self.history if
1820.                 h.score_b > h.score_a)
1821.             accept      = regressions == 0
1822.
1823.             return {
1824.                 "verdict":          "ACCEPT" if accept else
1825.                 "REJECT",
1826.                 "steps_evaluated":  len(self.history),
1827.                 "regressions_in_B": regressions,
1828.                 "divergence_trend": self._divergence_trend(),
1829.             }
1830.
1831.         def _divergence_trend(self) -> float:
1832.             if len(self.history) < 2:
1833.                 return 0.0
1834.
1835.             deltas = [abs(h.score_a - h.score_b) for h in
1836.                 self.history]
1837.
1838.             return statistics.mean(

```

```

1830.         deltas[i] - deltas[i - 1] for i in range(1,
1831.             len(deltas))
1832.     )
1833.     #
1834.     -----
1835.     # REPORT
1836.     #
1837.     -----
1838.     def report(self) -> Dict[str, float]:
1839.         if not self.history:
1840.             logger.info("report | empty")
1841.             return {}
1842.             r = {
1843.                 "avg_score_divergence":
1844.                     statistics.mean(abs(h.score_a - h.score_b)
1845.                                     for h
1846.                                     in self.history),
1847.                 "action_divergence_rate": statistics.mean(1 if
1848.                     h.action_a != h.action_b else 0
1849.                     for h
1850.                     in self.history),
1851.                 "energy_drift":
1852.                     statistics.mean(abs(h.energy_a - h.energy_b)
1853.                                     for h
1854.                                     in self.history),
1855.                 "coherence_drift":
1856.                     statistics.mean(abs(h.coherence_a - h.coherence_b)
1857.                                     for h
1858.                                     in self.history),
1859.                 "steps": len(self.history),
1860.             }
1861.             logger.info(f"report | {r}")
1862.             return r
1863.
1864.     def reset(self):
1865.         self.history.clear()
1866.         logger.info("reset")

```

```

1860. #
=====
1861. # PART W – SENTINEL CONFIG + RUNNER
1862. #
=====
1863.
1864. @dataclass
1865. class SentinelConfig:
1866.     strategy: str = "log_odds"
1867.     horizon: int = 36
1868.     stable_threshold: float = 1e-4
1869.     marginal_threshold: float = 1e-2
1870.     lyapunov_weight_overrides: Optional[Dict[str, float]] =
        None
1871.     energy_plot_path: str = "energy_plot.png"
1872.     classification_plot_path: str =
        "classification_timeline.png"
1873.     report_path: str = "stability_report.txt"
1874.
1875.
1876. def run_sentinel(
1877.     config: SentinelConfig,
1878.     baseline: Metrics,
1879.     cohort: CohortSummary,
1880.     delta_schedule: Callable[[int], TouchpointDelta],
1881. ) -> dict:
1882.     set_lyapunov_weights(config.lyapunov_weight_overrides)
1883.     model = (PredictiveReactionSandboxLogOdds() if
        config.strategy == "log_odds"
1884.             else PredictiveReactionSandbox())
1885.     monitor = WorkforceStabilityMonitor(baseline)
1886.     results = monitor.assess_future_stability(
1887.         model, config.horizon, cohort, delta_schedule,
1888.         config.stable_threshold, config.marginal_threshold,
1889.     )
1890.
1891.     StabilityDashboard.plot_energy_timeline(results["energies"],
        filename=config.energy_plot_path)
1892.
1893.     StabilityDashboard.plot_classification_timeline(results["stabil
        ity_classification"],

```

```

1893.         filename=config.classification_plot_path)
1894.     report_path = StabilityDashboard.build_report(
1895.         results["energies"],
1896.         results["stability_classification"],
1897.         filename=config.report_path,
1898.     )
1899.     return {**results, "artifacts": {"energy_plot":
1900.         config.energy_plot_path,
1901.         "classification_plot":
1902.         config.classification_plot_path,
1903.         "report": report_path}}
1904. #
1905. #
1906. #
1907. #
1908. #
1909. #
1910. #
1911. #
1912. #
1913. #
1914. #
1915. #
1916. #
1917. #
1918. #
1919. #
1920. #
1921. #
1922. #
1923. #
1924. #
1925. #
1926. #

```



```

1927.
1928. CANONICAL_SCENARIOS: Dict[str, Dict[str, Any]] = {
1929.     "save_restart_surge": {
1930.         "description": "SAVE repayment restart – high shock,
moderate confusion",
1931.         "baseline": _make_baseline(),
1932.         "cohort": _make_cohort(shock=0.75,
confusion=0.50, escalation=0.35),
1933.         "delta": TouchpointDelta(
1934.             update_id="SAVE-001",
channel=TouchpointChannel.IVR,
1935.             exposure_duration_secs=45.0,
friction_coefficient=0.6,
1936.             clarity_index=0.4, randomize_verification=False,
1937.         ),
1938.         "expected_regime": SystemRegime.SURGE,
1939.     },
1940.     "fraud_determinism_probe": {
1941.         "description": "Adversary attempts to raise
determinism index",
1942.         "baseline": _make_baseline(),
1943.         "cohort": _make_cohort(shock=0.3, confusion=0.7,
escalation=0.1),
1944.         "delta": TouchpointDelta(
1945.             update_id="FRAUD-001",
channel=TouchpointChannel.IVR,
1946.             exposure_duration_secs=10.0,
friction_coefficient=0.1,
1947.             clarity_index=0.9, randomize_verification=False,
1948.         ),
1949.         "expected_regime": SystemRegime.FRAUD_WAVE,
1950.     },
1951.     "queue_saturation_cascade": {
1952.         "description": "Back-office overload triggering queue
collapse",
1953.         "baseline": Metrics(abandon_rate=0.12,
ivr_reentry_rate=0.18,
1954.             transfer_rate=0.20, aht=520.0,
callback_rate=0.15,
1955.             backoffice_trigger=0.18,
determinism_index=0.85,

```

```

1956.                                     short_disconnect_rate=0.07,
    containment=0.68,
1957.                                     queue_depth=0.92),
1958.     "cohort":     _make_cohort(shock=0.6, confusion=0.4,
    escalation=0.4),
1959.     "delta": TouchpointDelta(
1960.         update_id="QUEUE-001",
    channel=TouchpointChannel.IVR,
1961.         exposure_duration_secs=30.0,
    friction_coefficient=0.5,
1962.         clarity_index=0.5, randomize_verification=False,
1963.     ),
1964.     "expected_regime": SystemRegime.SATURATED,
1965. },
1966.     "panic_emotional_escalation": {
1967.         "description": "High emotional state + high abandon -
    psychological safety priority",
1968.         "baseline":     _make_baseline(),
1969.         "cohort":     _make_cohort(shock=0.9, confusion=0.3,
    escalation=0.6),
1970.         "delta": TouchpointDelta(
1971.             update_id="PANIC-001",
    channel=TouchpointChannel.IVR,
1972.             exposure_duration_secs=90.0,
    friction_coefficient=0.8,
1973.             clarity_index=0.2, randomize_verification=True,
1974.         ),
1975.         "expected_regime": SystemRegime.PANIC,
1976.     },
1977.     "stable_baseline_nominal": {
1978.         "description": "Normal operations - verify no false
    positives",
1979.         "baseline":     _make_baseline(),
1980.         "cohort":     _make_cohort(shock=0.2, confusion=0.8,
    escalation=0.1),
1981.         "delta": TouchpointDelta(
1982.             update_id="STABLE-001",
    channel=TouchpointChannel.WEB_PORTAL,
1983.             exposure_duration_secs=10.0,
    friction_coefficient=0.1,
1984.             clarity_index=0.9, randomize_verification=False,
1985.         ),

```

```

1986.         "expected_regime": SystemRegime.STABLE,
1987.     },
1988. }
1989.
1990.
1991. def run_canonical_suite(sandbox: str = "log_odds") ->
    Dict[str, Dict]:
1992.     model = (PredictiveReactionSandboxLogOdds() if sandbox
    == "log_odds"
1993.             else PredictiveReactionSandbox())
1994.     detector = RegimeDetector()
1995.     results = {}
1996.
1997.     for name, scenario in CANONICAL_SCENARIOS.items():
1998.         post = model.generate_projected_metrics(
1999.             scenario["baseline"], scenario["cohort"],
    scenario["delta"])
2000.         profile = detector.detect(post, None, None)
2001.         results[name] = {
2002.             "description": scenario["description"],
2003.             "expected_regime":
    scenario["expected_regime"].value,
2004.             "detected_regime": profile.dominant_regime.value,
2005.             "regime_match": profile.dominant_regime ==
    scenario["expected_regime"],
2006.             "confidence": profile.confidence,
2007.             "post_metrics": post,
2008.         }
2009.
2010.     return results
2011.
2012.
2013. #
    =====
2014. # PART Y — EXAMPLE DRIVER
2015. #
    =====
2016.
2017. if __name__ == "__main__":
2018.     random.seed(42)
2019.
2020.     print("\n" + "=" * 60)

```

```

2021.     print("CANONICAL SCENARIO SUITE")
2022.     print("=" * 60)
2023.     suite = run_canonical_suite(sandbox="log_odds")
2024.     for name, r in suite.items():
2025.         match = "✓" if r["regime_match"] else "✗"
2026.         print(f"{match} {name}:
            expected={r['expected_regime']} "
2027.             f"detected={r['detected_regime']}
            conf={r['confidence']:.3f}")
2028.
2029.     print("\n" + "=" * 60)
2030.     print("BENCHMARK: baseline vs log_odds")
2031.     print("=" * 60)
2032.     base      = _make_baseline()
2033.     cohort    = _make_cohort()
2034.     delta_0 =
        CANONICAL_SCENARIOS["save_restart_surge"]["delta"]
2035.     harness = BenchmarkHarness()
2036.     for line in harness.compare(base, cohort, delta_0):
2037.         print(line)
2038.
2039.     print("\n" + "=" * 60)
2040.     print("LYAPUNOV STABILITY - log_odds / 24 steps")
2041.     print("=" * 60)
2042.
2043.     def delta_schedule(t: int) -> TouchpointDelta:
2044.         return TouchpointDelta(
2045.             update_id=f"T{t}", channel=TouchpointChannel.IVR,
2046.             exposure_duration_secs=5.0,
2047.             friction_coefficient=min(1.0, 0.2 + 0.01 * t),
2048.             clarity_index=max(0.1, 0.9 - 0.01 * t),
2049.             randomize_verification=(t % 12 == 0),
2050.         )
2051.
2052.     cfg      = SentinelConfig(strategy="log_odds", horizon=24)
2053.     results = run_sentinel(cfg, base, cohort, delta_schedule)
2054.     print(f"Artifacts: {results['artifacts']}")
2055.     print(f"Classifications:
        {results['stability_classification']}")
2056.
2057.     print("\n" + "=" * 60)

```

```

2058.     print("DUAL HARNESS - current vs proposed")
2059.     print("=" * 60)
2060.
2061.     store_a = EventStore(); bus_a = EventBus(store_a)
2062.     store_b = EventStore(); bus_b = EventBus(store_b)
2063.     ctx_a    = SentinelRunContext(run_id="A-001",
        timestamp=datetime.now())
2064.     ctx_b    = SentinelRunContext(run_id="B-001",
        timestamp=datetime.now())
2065.     dual     = DualSystemHarness(SentinelIVR(ctx_a, bus_a),
        SentinelIVR(ctx_b, bus_b), noise=0.01)
2066.
2067.     for sc_name in ["save_restart_surge",
        "queue_saturation_cascade",
2068.                    "panic_emotional_escalation",
        "stable_baseline_nominal"]:
2069.         sc    = CANONICAL_SCENARIOS[sc_name]
2070.         post  =
            PredictiveReactionSandboxLogOdds().generate_projected_metrics(
2071.                sc["baseline"], sc["cohort"], sc["delta"])
2072.         dual.step(PopulationType.SAVE, sc["baseline"], post,
            sc["cohort"])
2073.
2074.     print(f"Divergence report: {dual.report()}")
2075.     print(f"Governance verdict: {dual.verdict()}")
2076.
2077.     print("\n" + "=" * 60)
2078.     print("SOVEREIGN ENGINE - 12 training cycles")
2079.     print("=" * 60)
2080.     engine = SovereignEngine(
2081.         agents=[ConservativeAgent(), AggressiveAgent(),
            ReactiveAgent()],
2082.         noise_scale=3.0,
2083.     )
2084.     for cycle in range(12):
2085.         final_kpi = engine.run_cycle()
2086.         print(f"Cycle {cycle+1:02d} |
            regime={engine.regime:10s} | "
2087.               f"kpi={final_kpi:.2f} | "
            degraded_time={engine.degraded_time}")
2088.
2089.     print("\n" + "=" * 60)

```

```

2090.     print("REPLAY INTEGRITY")
2091.     print("=" * 60)
2092.     state = TraceReplayer(store_a).rebuild()
2093.     print(f"Evaluations logged: {len(state['evaluations'])}")
2094.     print(f"Event hashes:          {len(state['hashes'])}")
2095.
2096.
2097. #
      =====
2098. # SESSION CHANGE LOG
2099. #
      =====
2100. # [2026-05-18 09:57pm] Initial Sentinel IVR evaluation
      framework created.
2101. # [2026-05-18 10:08pm] Added regime detection architecture.
2102. # [2026-05-18 10:19pm] Added weighted regime scoring engine.
2103. # [2026-05-18 10:27pm] Added hysteresis stabilization system.
2104. # [2026-05-18 10:34pm] Added synthetic borrower architecture.
2105. # [2026-05-18 10:42pm] Added persistent architectural-intent
      export protocol.
2106. # [2026-05-18 10:55pm] Executed core code modification suite.
2107. # [2026-05-18 11:12pm] Added active mutation configuration
      engine.
2108. # [2026-05-18 11:35pm] Upgraded for high-scale 36,000-row
      population support.
2109. # [2026-05-18 11:54pm] Implemented omnichannel touchpoint
      contract refactor.
2110. # [2026-05-19 12:15am] Executed strategic adversarial
      hardening suite.
2111. #   - Bimodal masking fix (Simpson's Paradox guard)
2112. #   - Adversarial regime hijacking interlock
2113. # [2026-05-21] UNIFIED BLEND - sentinel_unified.py
2114. #   - Full integration of all eight modules, zero capability
      regression.
2115. #   - Canonical scenario library added (5 scenarios,
      regression runner).
2116. #   - Dual harness verdict() + divergence trend wired.
2117. #   - Event hash verification wired into
      TraceReplayer.rebuild().
2118. #   - Volatility proxy live: vol = min(2.0, world_err / 5.0).
2119. #   - Regime tolerance matrices restored and locked.
2120. #   - Bimodal masking fix and adversarial interlock present.

```

```
2121. #
=====
2122.
2123.
2124.
2125.
2126.
2127.
2128.
2129.
2130.
2131. Thoughts? What is it and how sophisticated is it?
2132.
2133. def evaluate_ivr(self, start_node: str):
2134.
2135.     if not self.context.ivr_graph:
2136.         return None
2137.
2138.     analyzer = IVRAnalyzer(self.context.ivr_graph)
2139.     metrics = analyzer.analyze(start_node)
2140.
2141.     return {
2142.         "avg_steps": metrics.avg_steps_to_resolution,
2143.         "avg_time": metrics.avg_time_to_resolution,
2144.         "abandonment": metrics.abandonment_rate,
2145.         "bottlenecks": metrics.bottleneck_nodes,
2146.         "entropy": metrics.path_entropy
2147.     }
2148.
2149.
2150.
2151. # =====
2152. # SENTINEL — ENTERPRISE EVENT-SOURCED GOVERNANCE SYSTEM
2153. # =====
2154.
2155. from dataclasses import dataclass, field
2156. from enum import Enum
2157. from datetime import datetime
2158. from typing import List, Dict, Any, Optional
2159. import hashlib
2160. import json
2161. import random
2162.
2163.
```

```
2164. # =====
2165. # CORE ENUMS
2166. # =====
2167.
2168. class PopulationType(Enum):
2169.     NORMAL = "normal"
2170.     SAVE = "save"
2171.     FRAUD = "fraud"
2172.
2173.
2174. class SentinelResult(Enum):
2175.     NEUTRAL = "neutral"
2176.     REGRESSIVE = "regressive"
2177.     RISK_INCREASING = "risk_increasing"
2178.
2179.
2180. # =====
2181. # CORE METRICS
2182. # =====
2183.
2184. @dataclass
2185. class Metrics:
2186.     containment: float
2187.     aht: float
2188.     callback_rate: float
2189.     abandon_rate: float
2190.     backoffice_trigger: float
2191.     determinism_index: float
2192.
2193.
2194. # =====
2195. # CONTEXT
2196. # =====
2197.
2198. @dataclass
2199. class SentinelRunContext:
2200.     run_id: str
2201.     timestamp: datetime
2202.     priority_profile: str
2203.     contaminated_conditions: List[str] = field(default_factory=list)
2204.     influencer_event_active: bool = False
2205.
2206.     def is_contaminated(self, exclusions: set[str]) -> bool:
2207.         return any(c in exclusions for c in self.contaminated_conditions)
```



```

2208.
2209.
2210. # =====
2211. # RESULTS
2212. # =====
2213.
2214. @dataclass
2215. class PopulationResult:
2216.     population: PopulationType
2217.     result: SentinelResult
2218.     rationale: str
2219.     run_id: str
2220.     timestamp: datetime
2221.
2222.
2223. # =====
2224. # EVENT SYSTEM
2225. # =====
2226.
2227. @dataclass(frozen=True)
2228. class Event:
2229.     event_type: str
2230.     run_id: str
2231.     timestamp: datetime
2232.     payload: Dict[str, Any]
2233.
2234.
2235. class EventStore:
2236.     def __init__(self):
2237.         self._events: List[Event] = []
2238.
2239.     def append(self, event: Event):
2240.         self._events.append(event)
2241.
2242.     def all(self) -> List[Event]:
2243.         return list(self._events)
2244.
2245.
2246. class EventBus:
2247.     def __init__(self, store: EventStore):
2248.         self.store = store
2249.
2250.     def emit(self, event: Event):
2251.         self.store.append(event)

```

```

2252.
2253.
2254. # =====
2255. # GOVERNANCE RULES
2256. # =====
2257.
2258. LEARNING_EXCLUSIONS = {
2259.     "ivr_port_saturation",
2260.     "hostile_automation",
2261.     "ato_evening_saturation"
2262. }
2263.
2264.
2265. # =====
2266. # GOVERNANCE ENGINE
2267. # =====
2268.
2269. class GovernanceEngine:
2270.
2271.     def __init__(self, context: SentinelRunContext):
2272.         self.context = context
2273.
2274.     def learning_allowed(self) -> bool:
2275.         return not self.context.is_contaminated(LEARNING_EXCLUSIONS)
2276.
2277.     def evaluate_fraud(self, baseline: Metrics, post: Metrics, run_id: str, ts: datetime):
2278.         delta = post.determinism_index - baseline.determinism_index
2279.
2280.         return PopulationResult(
2281.             population=PopulationType.FRAUD,
2282.             result=SentinelResult.RISK_INCREASING if delta > 0.05 else
                SentinelResult.NEUTRAL,
2283.             rationale=f"fraud_delta={delta:.3f}",
2284.             run_id=run_id,
2285.             timestamp=ts
2286.         )
2287.
2288.     def evaluate_borrower(self, pop: PopulationType, baseline: Metrics, post: Metrics,
                run_id: str, ts: datetime):
2289.
2290.         failures = []
2291.
2292.         if post.aht > baseline.aht:
2293.             failures.append("aht")

```

```

2294.
2295. if post.abandon_rate > baseline.abandon_rate:
2296.     failures.append("abandon")
2297.
2298. if post.backoffice_trigger > baseline.backoffice_trigger:
2299.     failures.append("backoffice")
2300.
2301. return PopulationResult(
2302.     population=pop,
2303.     result=SentinelResult.REGRESSIVE if failures else SentinelResult.NEUTRAL,
2304.     rationale=",".join(failures) if failures else "ok",
2305.     run_id=run_id,
2306.     timestamp=ts
2307. )
2308.
2309.
2310. # =====
2311. # ADVISORY ENGINE
2312. # =====
2313.
2314. class Advisor:
2315.
2316.     def __init__(self, context: SentinelRunContext):
2317.         self.context = context
2318.
2319.     def advisory(self, metrics: Metrics):
2320.
2321.         save_wave = (
2322.             metrics.abandon_rate > 0.10 or
2323.             metrics.backoffice_trigger > 0.17 or
2324.             len(self.context.contaminated_conditions) > 0
2325.         )
2326.
2327.         if save_wave:
2328.             return {
2329.                 "profile": "SAVE_WAVE",
2330.                 "directives": [
2331.                     "reduce IVR branching complexity",
2332.                     "stabilize messaging flow",
2333.                     "limit back-office escalation load"
2334.                 ]
2335.             }
2336.
2337.         return {

```

```

2338. "profile": "DEFAULT",
2339. "directives": ["optimize efficiency flow"]
2340. }
2341.
2342.
2343. # =====
2344. # ADVERSARIAL SIMULATION (LIGHTWEIGHT)
2345. # =====
2346.
2347. class AdversaryModel:
2348.
2349. def inject_noise(self, metrics: Metrics) -> Metrics:
2350. metrics.determinism_index += random.uniform(0, 0.08)
2351. return metrics
2352.
2353.
2354. # =====
2355. # DETERMINISTIC REPLAY ENGINE
2356. # =====
2357.
2358. class DeterministicReplay:
2359.
2360. def hash_event(self, event: Event) -> str:
2361. return hashlib.sha256(
2362. json.dumps(event.__dict__, default=str).encode()
2363. ).hexdigest()
2364.
2365.
2366. class TraceReplayer:
2367.
2368. def __init__(self, store: EventStore):
2369. self.store = store
2370.
2371. def rebuild(self) -> Dict[str, Any]:
2372.
2373. state = {
2374. "evaluations": [],
2375. "advisories": []
2376. }
2377.
2378. for e in self.store.all():
2379.
2380. if e.event_type == "EVAL_RESULT":
2381. state["evaluations"].append(e.payload)

```

```

2382.
2383. if e.event_type == "ADVISORY":
2384.     state["advisories"].append(e.payload)
2385.
2386. return state
2387.
2388.
2389. # =====
2390. # SENTINEL CORE ORCHESTRATOR
2391. # =====
2392.
2393. class Sentinel:
2394.
2395.     def __init__(self, context: SentinelRunContext, bus: EventBus):
2396.         self.context = context
2397.         self.bus = bus
2398.         self.engine = GovernanceEngine(context)
2399.         self.advisor = Advisor(context)
2400.
2401.     def evaluate(self, pop: PopulationType, baseline: Metrics, post: Metrics):
2402.
2403.         self.bus.emit(Event(
2404.             event_type="EVAL_START",
2405.             run_id=self.context.run_id,
2406.             timestamp=self.context.timestamp,
2407.             payload={"population": pop.value}
2408.         ))
2409.
2410.         if pop == PopulationType.FRAUD:
2411.             result = self.engine.evaluate_fraud(baseline, post, self.context.run_id,
                self.context.timestamp)
2412.         else:
2413.             result = self.engine.evaluate_borrower(pop, baseline, post, self.context.run_id,
                self.context.timestamp)
2414.
2415.         self.bus.emit(Event(
2416.             event_type="EVAL_RESULT",
2417.             run_id=self.context.run_id,
2418.             timestamp=self.context.timestamp,
2419.             payload={
2420.                 "population": pop.value,
2421.                 "result": result.result.value,
2422.                 "rationale": result.rationale
2423.             }

```

```
2424. ))
2425.
2426. return result
2427.
2428. def advise(self, metrics: Metrics):
2429.
2430.     advice = self.advisor.advisory(metrics)
2431.
2432.     self.bus.emit(Event(
2433.         event_type="ADVISORY",
2434.         run_id=self.context.run_id,
2435.         timestamp=self.context.timestamp,
2436.         payload=advice
2437.     ))
2438.
2439.     return advice
2440.
2441.
2442. # =====
2443. # EVENT STORE + BUS INITIALIZATION EXAMPLE
2444. # =====
2445.
2446. store = EventStore()
2447. bus = EventBus(store)
2448.
2449.
2450. # =====
2451. # SYSTEM NOW READY FOR:
2452. # - evaluation
2453. # - advisory
2454. # - replay
2455. # - audit reconstruction
2456. # =====
2457.
2458.
2459.
2460.
2461.
2462.
2463. William King
2464. Cell: 614-955-8951
2465.
2466.
2467. William King
```

2468. Cell: 614-955-8951
2469.
2470.
2471. ----- Forwarded message -----
2472. From: **King, William - ADVS** <WilliamKing@aidvantage.com>
2473. Date: Wed, May 20, 2026 at 5:09 PM
2474. Subject: sentinel (build process)
2475. To: William King <wking53214@gmail.com>
2476.
2477.
2478. ""
2479. sentinel.py
2480. Corrected, runnable module: baseline (nonlinear) + log-odds models, Lyapunov
stability, dashboard,:Corrected, runnable module: baseline (nonlinear) + log-odds
models, Lyapunov stability, dashboard, benchmark harness, and example driver.
2481. - energy_plot_baseline.png
2482. - classification_timeline_baseline.png
2483. - stability_report_baseline.txt
2484. - energy_plot_logodds.png
2485. - classification_timeline_logodds.png
2486. - stability_report_logodds.txt
2487. ""
2488.
2489. import os
2490. import math
2491. from dataclasses import dataclass
2492. from typing import Callable, Dict, List, Optional, Protocol, Tuple
2493.
2494. import numpy as np
2495. import matplotlib
2496. matplotlib.use('Agg') # Safe for headless environments (servers/containers)
2497. import matplotlib.pyplot as plt
2498.
2499. # =====
2500. # PART 2 — Utilities
2501. # =====
2502. def logit(p: float) -> float:
2503. ""
2504. Convert a probability into log-odds.
2505. $\text{logit}(p) = \ln(p / (1 - p))$
2506. ""
2507. $p = \min(\max(p, 1e-9), 1 - 1e-9)$
2508. return $\text{math.log}(p / (1 - p))$
2509.

```

2510. def inv_logit(x: float) -> float:
2511.     """
2512.     Convert log-odds back into a probability.
2513.      $p = 1 / (1 + e^{-x})$ 
2514.     """
2515.     return 1.0 / (1.0 + math.exp(-x))
2516.
2517. def soft_bound_sigmoid(x: float) -> float:
2518.     """
2519.     Smoothly maps any real value to [0, 1] using a logistic sigmoid.
2520.
2521.     Notes:
2522.     • For inputs close to 0, returns ~0.5; large positive → ~1, large negative → ~0.
2523.     """
2524.     return 1.0 / (1.0 + math.exp(-x))
2525.
2526. # =====
2527. # PART 3 — Data Models
2528. # =====
2529. @dataclass
2530. class RatePair:
2531.     """
2532.     Paired probability values:
2533.     unclipped : raw probability after model adjustments
2534.     p         : final usable probability (bounded/smoothed as needed)
2535.     """
2536.     unclipped: float
2537.     p: float
2538.
2539. @dataclass
2540. class Metrics:
2541.     """
2542.     Superset metrics used across stability analysis and behavioral sandboxes.
2543.
2544.     Baseline-required fields for Lyapunov:
2545.     abandon_rate      : Probability of abandonment
2546.     ivr_reentry_rate  : Probability of IVR re-entry
2547.     transfer_rate     : Probability of transfer
2548.     aht               : Average Handle Time (seconds)
2549.
2550.     Additional sandbox fields:
2551.     callback_rate
2552.     backoffice_trigger
2553.     determinism_index

```



```

2554.     repeat_auth_rate
2555.     containment
2556.     short_disconnect_rate
2557.     queue_depth
2558.
2559.     Diagnostics:
2560.     unclipped_abandon_rate
2561.     unclipped_callback_rate
2562.     unclipped_ivr_reentry_rate
2563.     """
2564.     abandon_rate: float = 0.0
2565.     ivr_reentry_rate: float = 0.0
2566.     transfer_rate: float = 0.0
2567.     aht: float = 0.0
2568.
2569.     # Extended sandbox fields
2570.     callback_rate: float = 0.0
2571.     backoffice_trigger: float = 0.0
2572.     determinism_index: float = 1.0
2573.     repeat_auth_rate: float = 0.0
2574.     containment: float = 0.0
2575.     short_disconnect_rate: float = 0.0
2576.     queue_depth: int = 0
2577.
2578.     # Diagnostics (optional)
2579.     unclipped_abandon_rate: Optional[float] = None
2580.     unclipped_callback_rate: Optional[float] = None
2581.     unclipped_ivr_reentry_rate: Optional[float] = None
2582.
2583. @dataclass
2584. class CohortSummary:
2585.     """
2586.     Aggregated cohort behavioral traits used by sandbox models.
2587.
2588.     Fields:
2589.     avg_abandonment_threshold : Higher → more patient users; lower → quicker to
        abandon.
2590.     avg_confusion_tolerance   : Higher → users tolerate more confusion; lower →
        re-entry & repeat auth increase.
2591.     avg_escalation_propensity : Higher → users are more likely to request
        transfers/escalation.
2592.     """
2593.     avg_abandonment_threshold: float = 0.5
2594.     avg_confusion_tolerance: float = 0.5

```

```

2595.     avg_escalation_propensity: float = 0.1
2596.
2597. @dataclass
2598. class TouchpointDelta:
2599.     """
2600.     Describes deltas introduced by a UX change or new touchpoint.
2601.
2602.     Fields:
2603.     friction_coefficient      : [0, 1+] higher increases AHT and abandonment in
        baseline model; impacts log-odds in refactor.
2604.     exposure_duration_secs   : Additional seconds added to interaction (e.g., new
        verification step).
2605.     randomize_verification   : If True, reduces determinism index (process is less
        predictable).
2606.     """
2607.     friction_coefficient: float = 0.0
2608.     exposure_duration_secs: float = 0.0
2609.     randomize_verification: bool = False
2610.
2611. # =====
2612. # PART 4 — Strategy Protocol
2613. # =====
2614. class BehaviorModel(Protocol):
2615.     def generate_projected_metrics(self, base: Metrics, summ: CohortSummary, d:
        TouchpointDelta) -> Metrics:
2616.         ...
2617.
2618. # =====
2619. # PART 5 — Baseline PredictiveReactionSandbox (Nonlinear)
2620. # =====
2621. class PredictiveReactionSandbox:
2622.     """
2623.     Original nonlinear predictive model.
2624.
2625.     Notes:
2626.     • Uses multiplicative scaling and sigmoid bounding via soft_bound_sigmoid.
2627.     • Serves as a "legacy" baseline for benchmarking against the log-odds model.
2628.     """
2629.
2630.     def generate_projected_metrics(self, base: Metrics, summ: CohortSummary, d:
        TouchpointDelta) -> Metrics:
2631.         # 1. Core drivers
2632.         aht = self._project_aht(base.aht, d)
2633.         abandonment = self._project_abandon_rate(base, summ, d)

```

```

2634.     reentry = self._project_reentry_rate(summ)
2635.     repeat_auth = self._project_repeat_auth_rate(summ)
2636.     transfer = self._project_transfer_rate(summ)
2637.     containment = self._project_containment(abandonment.p, transfer.p)
2638.
2639.     # 2. Second-order effects
2640.     callback = self._project_callback_rate(base, abandonment.unclipped)
2641.     backoffice = self._project_backoffice_trigger(base, reentry.p)
2642.     determinism = self._project_determinism_index(base, d)
2643.     short_disc = self._project_short_disconnect(base, summ)
2644.
2645.     # 3. Assemble
2646.     return Metrics(
2647.         containment=round(containment, 3),
2648.         aht=round(aht, 1),
2649.         callback_rate=round(callback.p, 3),
2650.         abandon_rate=round(abandonment.p, 3),
2651.         backoffice_trigger=round(backoffice, 3),
2652.         determinism_index=round(determinism, 3),
2653.         repeat_auth_rate=round(repeat_auth.p, 3),
2654.         ivr_reentry_rate=round(reentry.p, 3),
2655.         transfer_rate=round(transfer.p, 3),
2656.         short_disconnect_rate=round(short_disc, 3),
2657.         queue_depth=base.queue_depth,
2658.
2659.         # raw / unclipped
2660.         unclipped_abandon_rate=round(abandonment.unclipped, 4),
2661.         unclipped_callback_rate=round(callback.unclipped, 4),
2662.         unclipped_ivr_reentry_rate=round(reentry.unclipped, 4),
2663.     )
2664.
2665.     # ----- PRIVATE HELPERS -----
2666.     def _project_aht(self, base_aht: float, d: TouchpointDelta) -> float:
2667.         increase = d.exposure_duration_secs
2668.         if d.friction_coefficient > 0.5:
2669.             increase += d.friction_coefficient * 40
2670.         return base_aht + increase
2671.
2672.     def _project_abandon_rate(self, base: Metrics, summ: CohortSummary, d:
        TouchpointDelta) -> RatePair:
2673.         abandon_mult = 1 + (1 - summ.avg_abandonment_threshold) * 0.5
2674.         unclipped = base.abandon_rate * abandon_mult * (1 + d.friction_coefficient * 0.3)
2675.         # Bound with a centered sigmoid around 0.5
2676.         bounded = soft_bound_sigmoid(5 * (unclipped - 0.5))

```

```

2677.     return RatePair(unclipped=unclipped, p=bounded)
2678.
2679.     def _project_reentry_rate(self, summ: CohortSummary) -> RatePair:
2680.         unclipped = (1 - summ.avg_confusion_tolerance) * 0.40
2681.         bounded = soft_bound_sigmoid(5 * (unclipped - 0.5))
2682.         return RatePair(unclipped=unclipped, p=bounded)
2683.
2684.     def _project_repeat_auth_rate(self, summ: CohortSummary) -> RatePair:
2685.         unclipped = (1 - summ.avg_confusion_tolerance) * 0.25
2686.         bounded = soft_bound_sigmoid(5 * (unclipped - 0.5))
2687.         return RatePair(unclipped=unclipped, p=bounded)
2688.
2689.     def _project_transfer_rate(self, summ: CohortSummary) -> RatePair:
2690.         unclipped = summ.avg_escalation_propensity * 0.85
2691.         bounded = soft_bound_sigmoid(5 * (unclipped - 0.5))
2692.         return RatePair(unclipped=unclipped, p=bounded)
2693.
2694.     def _project_containment(self, p_abandon: float, p_transfer: float) -> float:
2695.         return max(0.0, 1.0 - p_abandon - p_transfer)
2696.
2697.     def _project_callback_rate(self, base: Metrics, unclipped_abandon: float) ->
RatePair:
2698.         unclipped = base.callback_rate * (1.1 if unclipped_abandon > 0.15 else 1.0)
2699.         bounded = soft_bound_sigmoid(5 * (unclipped - 0.5))
2700.         return RatePair(unclipped=unclipped, p=bounded)
2701.
2702.     def _project_backoffice_trigger(self, base: Metrics, p_reentry: float) -> float:
2703.         return soft_bound_sigmoid(5 * ((base.backoffice_trigger + (p_reentry * 0.15)) -
0.5))
2704.
2705.     def _project_determinism_index(self, base: Metrics, d: TouchpointDelta) -> float:
2706.         if d.randomize_verification:
2707.             return max(0.01, base.determinism_index - 0.40)
2708.         return base.determinism_index
2709.
2710.     def _project_short_disconnect(self, base: Metrics, summ: CohortSummary) ->
float:
2711.         abandon_mult = 1 + (1 - summ.avg_abandonment_threshold) * 0.5
2712.         return base.short_disconnect_rate * abandon_mult
2713.
2714.     # =====
2715.     # PART 6 — PredictiveReactionSandboxLogOdds (Improved)
2716.     # =====
2717.     class PredictiveReactionSandboxLogOdds:

```

```

2718.      """
2719.      Advanced behavior model using log-odds transformations.
2720.
2721.      Key motivations:
2722.      • Behavioral reactions tend to interact additively in log-odds space.
2723.      • Avoids unrealistic multiplicative scaling of probabilities.
2724.      • Produces more stable, smooth, and monotonic behavioral curves.
2725.      """
2726.
2727.      def generate_projected_metrics(self, base: Metrics, summ: CohortSummary, d:
        TouchpointDelta) -> Metrics:
2728.          # Primary behavior projections
2729.          aht = self._aht(base, d)
2730.          abandonment = self._abandon(base, summ, d)
2731.          reentry = self._reentry(summ)
2732.          repeat_auth = self._repeat_auth(summ)
2733.          transfer = self._transfer(summ)
2734.
2735.          # Dependent interactions
2736.          containment = self._containment(abandonment.p, transfer.p)
2737.          callback = self._callback(base, abandonment.p)
2738.          backoffice = self._backoffice(base, reentry.p)
2739.          determinism = self._determinism(base, d)
2740.          short_disc = self._short_disconnect(base, summ)
2741.
2742.          # Assemble final output
2743.          return Metrics(
2744.              containment=round(containment, 3),
2745.              aht=round(aht, 1),
2746.              callback_rate=round(callback.p, 3),
2747.              abandon_rate=round(abandonment.p, 3),
2748.              backoffice_trigger=round(backoffice, 3),
2749.              determinism_index=round(determinism, 3),
2750.              repeat_auth_rate=round(repeat_auth.p, 3),
2751.              ivr_reentry_rate=round(reentry.p, 3),
2752.              transfer_rate=round(transfer.p, 3),
2753.              short_disconnect_rate=round(short_disc, 3),
2754.              queue_depth=base.queue_depth,
2755.
2756.              unclipped_abandon_rate=round(abandonment.unclipped, 4),
2757.              unclipped_callback_rate=round(callback.unclipped, 4),
2758.              unclipped_ivr_reentry_rate=round(reentry.unclipped, 4),
2759.          )
2760.

```

```

2761.     # ----- PRIVATE HELPERS -----
2762.     def _aht(self, base: Metrics, d: TouchpointDelta) -> float:
2763.         friction_amp = d.friction_coefficient * 25
2764.         exposure = d.exposure_duration_secs
2765.         return base.aht + exposure + friction_amp
2766.
2767.     def _abandon(self, base: Metrics, summ: CohortSummary, d: TouchpointDelta) ->
        RatePair:
2768.         base_logit = logit(base.abandon_rate)
2769.         adjustment = (1 - summ.avg_abandonment_threshold) * 1.2 +
            d.friction_coefficient * 0.9
2770.         unclipped = inv_logit(base_logit + adjustment)
2771.         return RatePair(unclipped=unclipped, p=unclipped)
2772.
2773.     def _reentry(self, summ: CohortSummary) -> RatePair:
2774.         base = logit(0.10) # typical baseline
2775.         adj = (1 - summ.avg_confusion_tolerance) * 1.5
2776.         unclipped = inv_logit(base + adj)
2777.         return RatePair(unclipped=unclipped, p=unclipped)
2778.
2779.     def _repeat_auth(self, summ: CohortSummary) -> RatePair:
2780.         base = logit(0.05)
2781.         adj = (1 - summ.avg_confusion_tolerance) * 1.0
2782.         unclipped = inv_logit(base + adj)
2783.         return RatePair(unclipped=unclipped, p=unclipped)
2784.
2785.     def _transfer(self, summ: CohortSummary) -> RatePair:
2786.         base = logit(0.12)
2787.         adj = summ.avg_escalation_propensity * 1.3
2788.         unclipped = inv_logit(base + adj)
2789.         return RatePair(unclipped=unclipped, p=unclipped)
2790.
2791.     def _containment(self, abandon_p: float, transfer_p: float) -> float:
2792.         return max(0.0, 1.0 - abandon_p - transfer_p)
2793.
2794.     def _callback(self, base: Metrics, p_abandon: float) -> RatePair:
2795.         base_logit_cb = logit(base.callback_rate)
2796.         adj = (p_abandon - base.abandon_rate) * 2.0
2797.         unclipped = inv_logit(base_logit_cb + adj)
2798.         return RatePair(unclipped=unclipped, p=unclipped)
2799.
2800.     def _backoffice(self, base: Metrics, p_reentry: float) -> float:
2801.         base_logit_bo = logit(base.backoffice_trigger)
2802.         adj = p_reentry * 0.5

```

```

2803.     return inv_logit(base_logit_bo + adj)
2804.
2805.     def _determinism(self, base: Metrics, d: TouchpointDelta) -> float:
2806.         if d.randomize_verification:
2807.             return max(0.01, base.determinism_index - 0.30)
2808.         return base.determinism_index
2809.
2810.     def _short_disconnect(self, base: Metrics, summ: CohortSummary) -> float:
2811.         scale = 1 + (1 - summ.avg_abandonment_threshold) * 0.4
2812.         return base.short_disconnect_rate * scale
2813.
2814. # =====
2815. # PART 7 — Strategy Registry & Selector
2816. # =====
2817. MODEL_REGISTRY: Dict[str, Callable[[], BehaviorModel]] = {
2818.     "baseline": lambda: PredictiveReactionSandbox(),
2819.     "log_odds": lambda: PredictiveReactionSandboxLogOdds(),
2820. }
2821.
2822. def get_behavior_model(strategy: str) -> BehaviorModel:
2823.     """
2824.     Instantiate a behavior model by strategy name.
2825.
2826.     Valid values:
2827.     • "baseline"
2828.     • "log_odds"
2829.     """
2830.     key = strategy.strip().lower()
2831.     if key not in MODEL_REGISTRY:
2832.         raise ValueError(f"Unknown strategy '{strategy}'. Valid:
            {list(MODEL_REGISTRY.keys())}")
2833.     return MODEL_REGISTRY[key # instantiate via registry
2834.
2835. # =====
2836. # PART 8 — Lyapunov Stability Framework
2837. # =====
2838. LYAPUNOV_ENERGY_WEIGHTS = {
2839.     "abandon_rate": 1.0,
2840.     "ivr_reentry_rate": 1.0,
2841.     "transfer_rate": 1.0,
2842.     "normalized_aht": 1.0,
2843. }
2844.
2845. def set_lyapunov_weights(overrides: Optional[Dict[str, float]] = None) -> None:

```

```

2846. """
2847. Optionally override Lyapunov diagonal weights.
2848. """
2849. if not overrides:
2850.     return
2851. for k, v in overrides.items():
2852.     if k in LYAPUNOV_ENERGY_WEIGHTS and v > 0:
2853.         LYAPUNOV_ENERGY_WEIGHTS[k] = float(v)
2854.
2855. class LyapunovStabilityVerifier:
2856.     """
2857.     Lyapunov stability operations:
2858.     • Vectorized energy
2859.     • Batch evaluation
2860.     • ΔV (with V(baseline)=0)
2861.     • Classification
2862.     • Grid-search auto-tuning
2863.     """
2864.
2865.     @staticmethod
2866.     def compute_energy_state_vectorized(b: Metrics,
2867.                                         c: Metrics,
2868.                                         return_breakdown: bool = False):
2869.         """
2870.         Computes quadratic Lyapunov energy:
2871.
2872.         
$$V = \sum_i w_i (\Delta x_i)^2$$

2873.
2874.         where:
2875.         
$$\Delta x = [\Delta_{\text{abandon}}, \Delta_{\text{reentry}}, \Delta_{\text{transfer}}, \Delta_{\text{aht\_norm}}]$$

2876.
2877.         
$$\Delta_{\text{aht\_norm}} = (\text{AHT}_c - \text{AHT}_b) / (|\text{AHT}_b| + \epsilon)$$

2878.         """
2879.         delta = np.array([
2880.             c.abandon_rate - b.abandon_rate,
2881.             c.ivr_reentry_rate - b.ivr_reentry_rate,
2882.             c.transfer_rate - b.transfer_rate,
2883.             (c.aht - b.aht) / (abs(b.aht) + 1e-9),
2884.         ])
2885.
2886.         w = np.array([
2887.             max(LYAPUNOV_ENERGY_WEIGHTS["abandon_rate"], 1e-9),
2888.             max(LYAPUNOV_ENERGY_WEIGHTS["ivr_reentry_rate"], 1e-9),
2889.             max(LYAPUNOV_ENERGY_WEIGHTS["transfer_rate"], 1e-9),

```



```

2890.         max(LYAPUNOV_ENERGY_WEIGHTS["normalized_aht"], 1e-9),
2891.     ])
2892.
2893.     term_values = w * (delta ** 2)
2894.     energy = float(np.sum(term_values))
2895.
2896.     if return_breakdown:
2897.         return energy, {
2898.             "abandon_rate": float(term_values[0]),
2899.             "ivr_reentry_rate": float(term_values[1]),
2900.             "transfer_rate": float(term_values[2]),
2901.             "normalized_aht": float(term_values[3]),
2902.         }
2903.
2904.     return energy
2905.
2906.     @staticmethod
2907.     def compute_energy_batch(baseline: Metrics, metrics_list: List[Metrics]) ->
        List[float]:
2908.         """
2909.         Compute energy values across a sequence of Metrics points.
2910.         """
2911.         return [
2912.             LyapunovStabilityVerifier.compute_energy_state_vectorized(baseline, m)
2913.             for m in metrics_list
2914.         ]
2915.
2916.     @staticmethod
2917.     def delta_energy(b: Metrics, c: Metrics) -> dict:
2918.         """
2919.         Computes  $\Delta V = V(c) - V(b)$ . For this formulation  $V(b) = 0$ , so  $\Delta V = V(c)$ .
2920.         """
2921.         Vb = LyapunovStabilityVerifier.compute_energy_state_vectorized(b, b)
2922.         Vc = LyapunovStabilityVerifier.compute_energy_state_vectorized(b, c)
2923.         return {"V_baseline": Vb, "V_current": Vc, "delta_V": Vc - Vb}
2924.
2925.     @staticmethod
2926.     def classify_stability(energy: float,
2927.                           stable_threshold: float = 1e-4,
2928.                           marginal_threshold: float = 1e-2) -> str:
2929.         """
2930.         Classify the state based on Lyapunov energy magnitude.
2931.
2932.         Rules:

```

```

2933.         • energy <= stable_threshold           → "stable"
2934.         • stable_threshold < energy <= marginal_threshold → "marginal"
2935.         • energy > marginal_threshold           → "unstable"
2936.     """
2937.     if energy <= stable_threshold:
2938.         return "stable"
2939.     if energy <= marginal_threshold:
2940.         return "marginal"
2941.     return "unstable"
2942.
2943.     @staticmethod
2944.     def auto_tune_weights(baseline: Metrics,
2945.                           training_data: List[Metrics],
2946.                           weight_grid: Tuple[float, ...] = (0.1, 1.0, 2.0, 5.0)) -> dict:
2947.     """
2948.     Grid-search tuner minimizing total energy across a training dataset.
2949.
2950.     Returns:
2951.         {
2952.         "best_weights": (wa, wr, wt, wn),
2953.         "best_score": float
2954.         }
2955.     """
2956.     best_weights = None
2957.     best_score = float("inf")
2958.
2959.     for wa in weight_grid:
2960.         for wr in weight_grid:
2961.             for wt in weight_grid:
2962.                 for wn in weight_grid:
2963.
2964.                     LYAPUNOV_ENERGY_WEIGHTS["abandon_rate"] = wa
2965.                     LYAPUNOV_ENERGY_WEIGHTS["ivr_reentry_rate"] = wr
2966.                     LYAPUNOV_ENERGY_WEIGHTS["transfer_rate"] = wt
2967.                     LYAPUNOV_ENERGY_WEIGHTS["normalized_aht"] = wn
2968.
2969.                     batch_energy = sum(
2970.                         LyapunovStabilityVerifier.compute_energy_state_vectorized(baseline, m)
2971.                         for m in training_data
2972.                     )
2973.
2974.                     if batch_energy < best_score:
2975.                         best_score = batch_energy

```

```

2976.             best_weights = (wa, wr, wt, wn)
2977.
2978.     # Restore optimal weights
2979.     if best_weights is not None:
2980.         wa, wr, wt, wn = best_weights
2981.         LYAPUNOV_ENERGY_WEIGHTS["abandon_rate"] = wa
2982.         LYAPUNOV_ENERGY_WEIGHTS["ivr_reentry_rate"] = wr
2983.         LYAPUNOV_ENERGY_WEIGHTS["transfer_rate"] = wt
2984.         LYAPUNOV_ENERGY_WEIGHTS["normalized_aht"] = wn
2985.
2986.     return {"best_weights": best_weights, "best_score": best_score}
2987.
2988. # =====
2989. # PART 9 — Workforce Stability Monitor
2990. # =====
2991. class WorkforceStabilityMonitor:
2992.     """
2993.     Integrates prediction pipelines with Lyapunov stability evaluation.
2994.
2995.     Workflow:
2996.     baseline → delta_schedule(t) → behavior_model → metrics_t
2997.     metrics_t → Lyapunov energy → classification
2998.     """
2999.
3000.     def __init__(self, baseline: Metrics):
3001.         self.baseline = baseline
3002.
3003.     def assess_future_stability(self,
3004.                                behavior_model: BehaviorModel,
3005.                                horizon: int,
3006.                                summ: CohortSummary,
3007.                                delta_schedule: Callable[[int], TouchpointDelta],
3008.                                stable_threshold: float = 1e-4,
3009.                                marginal_threshold: float = 1e-2) -> dict:
3010.         """
3011.         Predict horizon-step future metrics, compute energy trajectory, and assign
3012.         classifications.
3013.
3014.         Returns:
3015.         {
3016.             "predictions": List[Metrics],
3017.             "energies": List[float],
3018.             "delta_V": List[float],
3019.             "stability_classification": List[str]

```

```

3019.         }
3020.     """
3021.     predictions: List[Metrics] = []
3022.     for t in range(horizon):
3023.         d = delta_schedule(t)
3024.         m = behavior_model.generate_projected_metrics(self.baseline, summ, d)
3025.         predictions.append(m)
3026.
3027.     energies = LyapunovStabilityVerifier.compute_energy_batch(self.baseline,
        predictions)
3028.
3029.     classifications = [
3030.         LyapunovStabilityVerifier.classify_stability(e, stable_threshold,
        marginal_threshold)
3031.         for e in energies
3032.     ]
3033.
3034.     return {
3035.         "predictions": predictions,
3036.         "energies": energies,
3037.         "delta_V": energies, # Baseline  $V=0 \rightarrow \Delta V = V$ 
3038.         "stability_classification": classifications,
3039.     }
3040.
3041. # =====
3042. # PART 10 — Stability Dashboard (Plots + Reporting)
3043. # =====
3044. class StabilityDashboard:
3045.     @staticmethod
3046.     def plot_energy_timeline(energies: List[float], filename: str = "energy_plot.png"):
3047.         plt.figure(figsize=(10, 4))
3048.         plt.plot(energies, label="Energy V")
3049.         plt.title("Lyapunov Energy Over Time")
3050.         plt.xlabel("Time Step")
3051.         plt.ylabel("Energy V")
3052.         plt.legend()
3053.         plt.tight_layout()
3054.         plt.savefig(filename)
3055.         plt.close()
3056.
3057.     @staticmethod
3058.     def plot_classification_timeline(classifications: List[str], filename: str =
        "classification_timeline.png"):
3059.         mapping = {"stable": 0, "marginal": 1, "unstable": 2}

```

```

3060.     y = [mapping.get(c, 1) for c in classifications]
3061.
3062.     plt.figure(figsize=(10, 4))
3063.     plt.step(range(len(y)), y, where='mid')
3064.     plt.yticks([0, 1, 2], ["stable", "marginal", "unstable"])
3065.     plt.title("Stability Classification Timeline")
3066.     plt.xlabel("Time Step")
3067.     plt.ylabel("Classification")
3068.     plt.tight_layout()
3069.     plt.savefig(filename)
3070.     plt.close()
3071.
3072.     @staticmethod
3073.     def build_report(energies: List[float], classifications: List[str], filename: str =
        "stability_report.txt") -> str:
3074.         report = "Stability Analytics Report\n" + "=" * 40 + "\n"
3075.         for i, (e, c) in enumerate(zip(energies, classifications)):
3076.             report += f"Step {i}: Energy={e:.6f}, State={c}\n"
3077.         with open(filename, "w") as f:
3078.             f.write(report)
3079.         return filename
3080.
3081.     # =====
3082.     # PART 11 — Benchmark Harness (Optional)
3083.     # =====
3084.     class BenchmarkHarness:
3085.         def __init__(self, old_model: BehaviorModel, new_model: BehaviorModel):
3086.             self.old = old_model
3087.             self.new = new_model
3088.
3089.         def compare(self, base: Metrics, summ: CohortSummary, d: TouchpointDelta) ->
            List[str]:
3090.             old_m = self.old.generate_projected_metrics(base, summ, d)
3091.             new_m = self.new.generate_projected_metrics(base, summ, d)
3092.
3093.             fields = [
3094.                 "aht",
3095.                 "abandon_rate",
3096.                 "callback_rate",
3097.                 "containment",
3098.                 "transfer_rate",
3099.                 "ivr_reentry_rate",
3100.                 "repeat_auth_rate",
3101.                 "backoffice_trigger",

```

```

3102.         "short_disconnect_rate",
3103.         "determinism_index",
3104.     ]
3105.
3106.     lines = ["=== BENCHMARK: OLD vs NEW ==="]
3107.     for f in fields:
3108.         old_val = getattr(old_m, f, None)
3109.         new_val = getattr(new_m, f, None)
3110.         if isinstance(old_val, (int, float)) and isinstance(new_val, (int, float)):
3111.             lines.append(f"{f:24s} | old={old_val:.4f} | new={new_val:.4f}")
3112.         else:
3113.             lines.append(f"{f:24s} | old={old_val} | new={new_val}")
3114.     return lines
3115.
3116. # =====
3117. # PART 12 — Sentinel Config & Runner
3118. # =====
3119. @dataclass
3120. class SentinelConfig:
3121.     strategy: str = "baseline" # "baseline" | "log_odds"
3122.     horizon: int = 36
3123.     stable_threshold: float = 1e-4
3124.     marginal_threshold: float = 1e-2
3125.     lyapunov_weight_overrides: Optional[Dict[str, float]] = None
3126.
3127.     # Output files
3128.     energy_plot_path: str = "energy_plot.png"
3129.     classification_plot_path: str = "classification_timeline.png"
3130.     report_path: str = "stability_report.txt"
3131.
3132. def run_sentinel(config: SentinelConfig,
3133.                 baseline: Metrics,
3134.                 cohort: CohortSummary,
3135.                 delta_schedule: Callable[[int], TouchpointDelta]) -> dict:
3136.     """
3137.     Execute the sentinel pipeline:
3138.     • Select behavior model
3139.     • Apply Lyapunov stability monitor
3140.     • Produce plots and text report
3141.     """
3142.     # Strategy selection
3143.     model = get_behavior_model(config.strategy)
3144.
3145.     # Optional Lyapunov weight overrides

```

```

3146.     set_lyapunov_weights(config.lyapunov_weight_overrides)
3147.
3148.     # Stability evaluation
3149.     monitor = WorkforceStabilityMonitor(baseline)
3150.     results = monitor.assess_future_stability(
3151.         behavior_model=model,
3152.         horizon=config.horizon,
3153.         summ=cohort,
3154.         delta_schedule=delta_schedule,
3155.         stable_threshold=config.stable_threshold,
3156.         marginal_threshold=config.marginal_threshold
3157.     )
3158.
3159.     # Diagnostics
3160.     StabilityDashboard.plot_energy_timeline(results["energies"],
        filename=config.energy_plot_path)
3161.     StabilityDashboard.plot_classification_timeline(results["stability_classification"],
        filename=config.classification_plot_path)
3162.
3163.     report_path = StabilityDashboard.build_report(results["energies"],
        results["stability_classification"],
3164.         filename=config.report_path)
3165.
3166.
3167.     # Return results + artifact paths
3168.     return {
3169.         **results,
3170.         "artifacts": {
3171.             "energy_plot": config.energy_plot_path,
3172.             "classification_plot": config.classification_plot_path,
3173.             "report": report_path
3174.         }
3175.     }
3176.
3177. # =====
3178. # PART 13 — Example Driver (Optional)
3179. # =====
3180. if __name__ == "__main__":
3181.     # --- Baseline scenario setup ---
3182.     baseline_metrics = Metrics(
3183.         abandon_rate=0.08,
3184.         ivr_reentry_rate=0.10,
3185.         transfer_rate=0.12,
3186.         aht=420.0, # seconds
3187.
3188.         callback_rate=0.06,

```

```

3189.     backoffice_trigger=0.04,
3190.     determinism_index=0.9,
3191.     short_disconnect_rate=0.03,
3192.     queue_depth=25,
3193. )
3194.
3195.     cohort_summary = CohortSummary(
3196.         avg_abandonment_threshold=0.6,
3197.         avg_confusion_tolerance=0.65,
3198.         avg_escalation_propensity=0.20,
3199.     )
3200.
3201.     def delta_schedule(t: int) -> TouchpointDelta:
3202.         # Mildly increasing friction and occasional randomization
3203.         return TouchpointDelta(
3204.             friction_coefficient=min(1.0, 0.2 + 0.01 * t),
3205.             exposure_duration_secs=5.0,
3206.             randomize_verification=(t % 12 == 0)
3207.         )
3208.
3209.     # --- Benchmark: old vs new sandbox (single step) ---
3210.     harness = BenchmarkHarness(PredictiveReactionSandbox(),
3211.                               PredictiveReactionSandboxLogOdds())
3211.     comparison_lines = harness.compare(baseline_metrics, cohort_summary,
3212.                                       delta_schedule(5))
3212.     print("\n".join(comparison_lines))
3213.
3214.     # --- Run sentinel with baseline strategy ---
3215.     cfg_baseline = SentinelConfig(
3216.         strategy="baseline",
3217.         horizon=24,
3218.         energy_plot_path="energy_plot_baseline.png",
3219.         classification_plot_path="classification_timeline_baseline.png",
3220.         report_path="stability_report_baseline.txt"
3221.     )
3222.     results_baseline = run_sentinel(cfg_baseline, baseline_metrics,
3223.                                    cohort_summary, delta_schedule)
3223.     print(f"\n[Baseline] Saved artifacts: {results_baseline['artifacts']}")
3224.
3225.     # --- Run sentinel with log-odds strategy (second optional model) ---
3226.     cfg_log_odds = SentinelConfig(
3227.         strategy="log_odds",
3228.         horizon=24,
3229.         energy_plot_path="energy_plot_logodds.png",

```



```
3230.     classification_plot_path="classification_timeline_logodds.png",
3231.     report_path="stability_report_logodds.txt"
3232. )
3233.     results_log_odds = run_sentinel(cfg_log_odds, baseline_metrics,
    cohort_summary, delta_schedule)
3234.     print(f"[Log-Odds] Saved artifacts: {results_log_odds['artifacts']}")
3235.
3236.     # --- Optional: show file sizes for convenience ---
3237.     for f in [
3238.         "energy_plot_baseline.png",
3239.         "classification_timeline_baseline.png",
3240.         "stability_report_baseline.txt",
3241.         "energy_plot_logodds.png",
3242.         "classification_timeline_logodds.png",
3243.         "stability_report_logodds.txt",
3244.     ]:
3245.         if os.path.exists(f):
3246.             print(f"{f}: {os.path.getsize(f)} bytes")
3247.         else:
3248.             print(f"Missing: {f}")
3249.
3250. Usage:
3251.     python sentinel.py
3252.
```